

Intermediate Machine Learning: Assignment 2

Deadline

Assignment 2 is due Thursday, October 9 by 11:59 pm. Late work will not be accepted as per the course policies (see the syllabus on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a .pdf on Gradescope. To convert your notebook to a .pdf while preserving the cell structure without truncating the output, you can convert to .html using [this notebook](#). Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

Topics

- Convolutional neural nets
- Gaussian processes
- Double descent

Please note that while the problems may look long, we give you substantial starter code to make the work manageable. The course staff members are available to help during office hours!

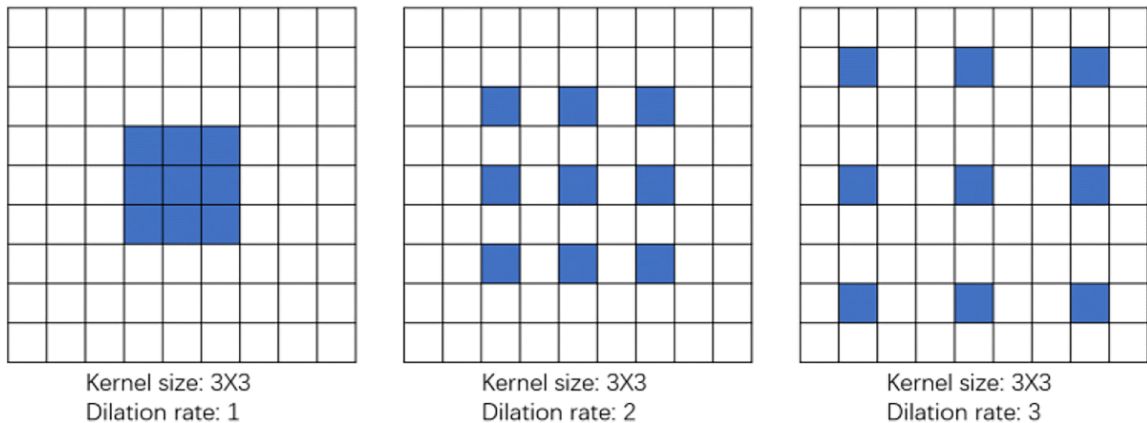
AI Use Acknowledgement

I used GPT-5 for assistance with these problems. Mostly, it was used to clean up and comment code, as well as for troubleshooting ideas.

Problem 1: Dilation elation (35 points)

Overview

A dilated convolution (also known as atrous convolution) inserts gaps between kernel entries, so that a $k \times k$ filter "jumps" by a dilation rate d . A dilation rate of $d = 1$ is the regular convolution. CNN dilation can be adjusted using parameter d as shown in the figure below.



Dilated convolution captures a wider area of the input, allowing for potentially better context modeling without increased computational cost. It expands the effective "receptive field" (the area of the filter) without the need for larger kernels or more model parameters; dilated convolution can also reduce blurring. One of the most successful uses of dilated convolution was in the first AlphaFold paper, one of the earliest CNN-based attempts to predict the structure of proteins. For a glimpse of the full architecture and its details you may wish to take a look at <https://www.nature.com/articles/s41586-019-1923-7>

In this problem, we use dilated convolution to analyze MRI images to detect the presence of certain types of brain tumors. Cues about tumor type can include texture information (enhanced rim, necrotic core) and global characteristics (edema extent, mass effect, midline shift). Dilation expands the effective receptive field while keeping high-resolution features available for downstream neural network layers that make the multi-class decision. This approach is also helps address the problem of "anisotropy" where some characteristics are directionally dependent, by trying to incorporate a larger sub image in the kernel.

In the first part of this problem, we implement a regular CNN. Then, your job will be to implement a dilated CNN, and to choose the dilation parameters so that the model is as accurate as you can make it. Finally, we use Monte Carlo dropout as a way to estimate the uncertainty in the predictions.

Dilated convolution in one and two dimensions

Symbols (what each letter means):

- x : the **input** (a 1D signal for the first formula; a 2D image for the second).
- y : the **output feature map** after convolution (same shape type as x).
- $w \in \mathbb{R}^k$: a 1D **kernel/filter** of length k .
- $W \in \mathbb{R}^{k \times k}$: a 2D **kernel/filter** of size $k \times k$.
- k : **kernel size** (e.g., $k = 3$ for a 3×3 kernel).
- d : **dilation rate** (how far apart the kernel taps are; $d = 1$ is a standard convolution).
- i, j : spatial **indices** of the output y (row/column).
- m, u, v : **indices inside the kernel** (1D: m , 2D: u for rows, v for cols).
- $u_c = \lfloor k/2 \rfloor, v_c = \lfloor k/2 \rfloor$: the **kernel center** offsets (so the kernel is centered).
- We assume **stride** = 1 and appropriate padding so indices are valid.

1D dilated convolution of a signal x with kernel $w \in \mathbb{R}^k$ and dilation $d \in \mathbb{N}$:

$$y[i] = \sum_{m=0}^{k-1} w[m] x(i + d m).$$

2D dilated convolution of an image x with kernel $W \in \mathbb{R}^{k \times k}$ and dilation d (stride 1).
Let $u_c = \lfloor k/2 \rfloor$ and $v_c = \lfloor k/2 \rfloor$ denote the kernel center:

$$y[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[u, v] x(i + d(u - u_c), j + d(v - v_c)).$$

Effective receptive field of a $k \times k$ kernel with dilation d :

$$k_{\text{eff}} = k + (k - 1)(d - 1).$$

For example, for $k = 3$ and $d = 3$, we get $k_{\text{eff}} = 7$.

(When $d = 1$, this reduces to the standard convolution.)

Loading the MRI data

The data used in this problem is cropped from this Kaggle repository

<https://www.kaggle.com/masoudnickparvar/brain-tumor-mri-dataset>.

We only use the data corresponding to glioma tumors and non-tumor images. You can access the data in a zip file at <https://github.com/YData123/sds365-fa25/tree/main/assignments/assn2/>. You will need to upload the data to your Google Drive.

The cell below gives Colab access to your Google Drive.

```
In [1]: # Please do not edit this cell.
# Imports
import os, random, time, math, keras, json
```

```

from glob import glob
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras import layers as L, models as M

# (Colab) Mount Google Drive
from google.colab import drive
if not os.path.ismount("/content/drive"):
    drive.mount("/content/drive")

```

Mounted at /content/drive

Now that we mounted Google Drive let's try visualizing images in the training data for the tumor and non-tumor classes.

```

In [2]: # Do not edit this block.
# Point to the MRI/ folder that contains Training/ and Testing/
ROOT = "/content/drive/MyDrive/kaggle-MRI"

TRAIN_DIR = os.path.join(ROOT, "Training")
TEST_DIR = os.path.join(ROOT, "Testing")
CLASS_NAMES = ["glioma", "notumor"] # expected two folders inside both train

# Validation of directory structure
def assert_dir(p):
    if not os.path.isdir(p):
        raise FileNotFoundError(f"Expected directory not found: {p}")

assert_dir(ROOT)
assert_dir(TRAIN_DIR)
assert_dir(TEST_DIR)

# Load image as grayscale for consistent viewing
def load_img_gray(path):
    img = Image.open(path).convert("L") # force grayscale
    return img

# List image files in a folder
def count_images(folder):
    exts = ("*.jpg", "*.jpeg")
    files = []
    for e in exts:
        files.extend(glob(os.path.join(folder, e)))
    return sorted(files)

```

The data we have requires pre-processing in order to be fed to any neural networks. The inconsistencies in the size as you see in the above output becomes problematic during feeding the data to the network. In this block, we strive to solve that issue.

```

In [3]: # Helper functions: letterbox pad to square (no cropping) + resize to 160x160

IMG_SIZE = 160 # target size used throughout the assignment

def letterbox_to_square(img: Image.Image, fill=0) -> Image.Image:
    """
    Pads the image to a square using 'fill' (black), centered, without distortion.
    Keeps original content intact (no cropping).
    """
    if img.mode not in ("L", "RGB"):
        img = img.convert("L")
    w, h = img.size
    s = max(w, h)
    canvas = Image.new(img.mode, (s, s), color=fill)
    canvas.paste(img, ((s - w)//2, (s - h)//2))
    return canvas

def preprocess_image(img: Image.Image, size: int = IMG_SIZE) -> Image.Image:
    """Letterbox pad -> resize to (size, size)."""
    sq = letterbox_to_square(img, fill=0)
    return sq.resize((size, size), resample=Image.BILINEAR)

# Visualize ORIGINAL vs PREPROCESSED (letterboxed) for first training image
fig, axes = plt.subplots(len(CLASS_NAMES), 2, figsize=(6, 3*len(CLASS_NAMES)))
if len(CLASS_NAMES) == 1:
    axes = np.array([axes]) # ensure 2D array indexing

for i, c in enumerate(CLASS_NAMES):
    class_dir = os.path.join(TRAIN_DIR, c)
    files = count_images(class_dir)
    ax_orig, ax_proc = axes[i, 0], axes[i, 1]
    if not files:
        ax_orig.axis("off"); ax_proc.axis("off")
        ax_proc.set_title(f"No images found for {c}")
        continue

    p = files[0]
    img = load_img_gray(p) # original grayscale
    proc = preprocess_image(img) # padded + resized 160x160

    ax_orig.imshow(img, cmap="gray"); ax_orig.axis("off")
    ax_orig.set_title(f"{c} (original {img.size[0]}x{img.size[1]})", fontsize=12)

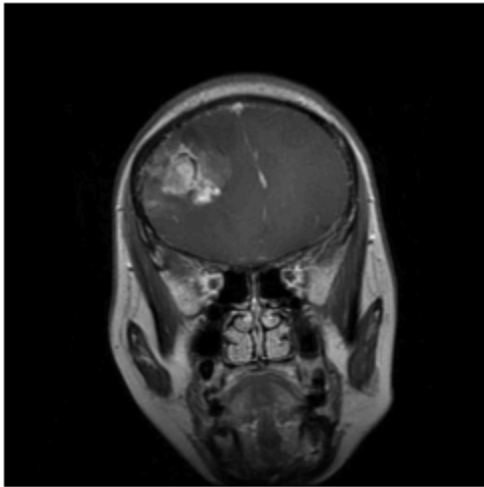
    ax_proc.imshow(proc, cmap="gray"); ax_proc.axis("off")
    ax_proc.set_title(f"{c} (letterbox -> {IMG_SIZE}x{IMG_SIZE})", fontsize=12)

plt.suptitle("Sanity check: letterbox padding avoids cropping", fontsize=12)
plt.tight_layout()
plt.show()

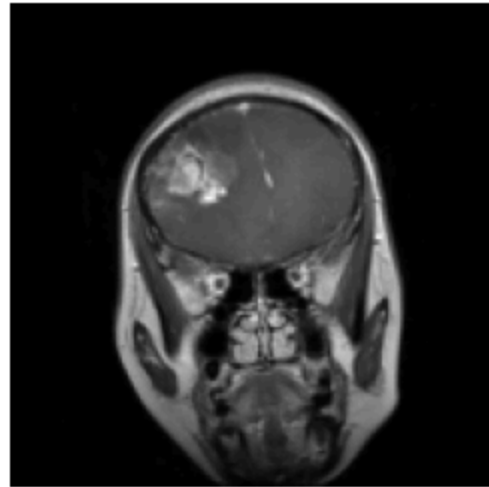
```

Sanity check: letterbox padding avoids cropping

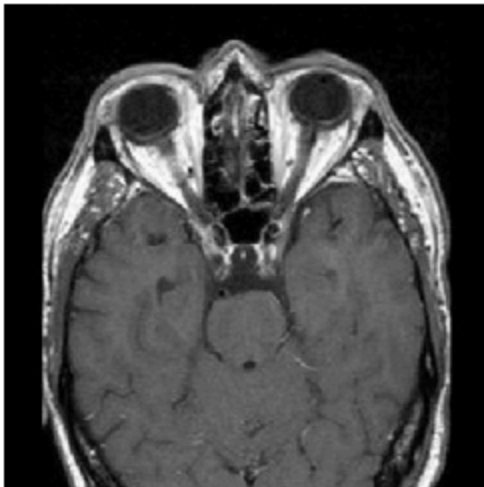
glioma (original 512×512)



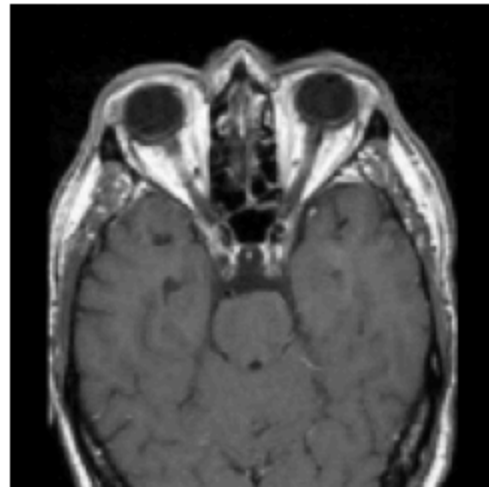
glioma (letterbox → 160×160)



notumor (original 350×350)



notumor (letterbox → 160×160)



In [4]: *# Gather file paths & labels; split TRAIN into train/validation (TEST untouched)*

```
def list_images_with_labels(root_dir, classes):
    paths, labels = [], []
    for ci, cname in enumerate(classes):
        cdir = os.path.join(root_dir, cname)
        for p in count_images(cdir):
            paths.append(p); labels.append(ci)
    return np.array(paths), np.array(labels)

# load TRAIN and TEST file lists
train_paths, train_labels = list_images_with_labels(TRAIN_DIR, CLASS_NAMES)
test_paths, test_labels = list_images_with_labels(TEST_DIR, CLASS_NAMES)

# stratified split of TRAIN -> (train, val)
VAL_FRACTION = 0.20
tr_paths, val_paths, tr_labels, val_labels = train_test_split(
    train_paths, train_labels,
    test_size=VAL_FRACTION,
    random_state=42,
```

```

    stratify=train_labels
)

print(f"Counts -> train: {len(tr_paths)}, val: {len(val_paths)}, test: {len(

```

Counts -> train: 2332, val: 584, test: 705

In [5]: *# TensorFlow input pipeline with the SAME preprocessing:*
grayscale -> letterbox to square (no crop) -> resize to IMG_SIZE -> per-im

```

def tf_letterbox_resize(img, target=IMG_SIZE):
    shape = tf.shape(img)
    h, w = shape[0], shape[1]
    s = tf.maximum(h, w)
    pad_top = (s - h) // 2
    pad_bottom = s - h - pad_top
    pad_left = (s - w) // 2
    pad_right = s - w - pad_left
    img_sq = tf.pad(img, [[pad_top, pad_bottom], [pad_left, pad_right], [0, 0]])
    img_rs = tf.image.resize(img_sq, (target, target), method="bilinear")
    return img_rs

def tf_per_image_zscore(img):
    mean = tf.reduce_mean(img)
    std = tf.math.reduce_std(img)
    return (img - mean) / tf.maximum(std, 1e-6)

def load_and_preprocess(path, label, augment=False):
    img_bytes = tf.io.read_file(path)
    img = tf.io.decode_image(img_bytes, channels=1, expand_animations=False)
    img = tf.image.convert_image_dtype(img, tf.float32)
    img = tf_letterbox_resize(img, target=IMG_SIZE)

    if augment:
        img = tf.image.random_flip_left_right(img)
        img = tf.image.random_brightness(img, max_delta=0.10)
        img = tf.image.random_contrast(img, lower=0.9, upper=1.1)

    img = tf_per_image_zscore(img)
    label = tf.cast(label, tf.float32)
    label = tf.expand_dims(label, axis=-1) # (1,)
    return img, label

def make_dataset(paths, labels, batch_size=32, shuffle=False, augment=False):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(paths), seed=42, reshuffle_each_iter
        ds = ds.map(lambda p, y: load_and_preprocess(p, y, augment=augment),
                    num_parallel_calls=tf.data.AUTOTUNE)
        ds = ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    return ds

BATCH_SIZE = 16
ds_train = make_dataset(tr_paths, tr_labels, batch_size=BATCH_SIZE, shuffle=True)
ds_val = make_dataset(val_paths, val_labels, batch_size=BATCH_SIZE, shuffle=False)
ds_test = make_dataset(test_paths, test_labels, batch_size=BATCH_SIZE, shuffle=False)

```



```
for xb, yb in ds_train.take(1):
    print("Train batch:", xb.shape, yb.shape)
```

Train batch: (16, 160, 160, 1) (16, 1)

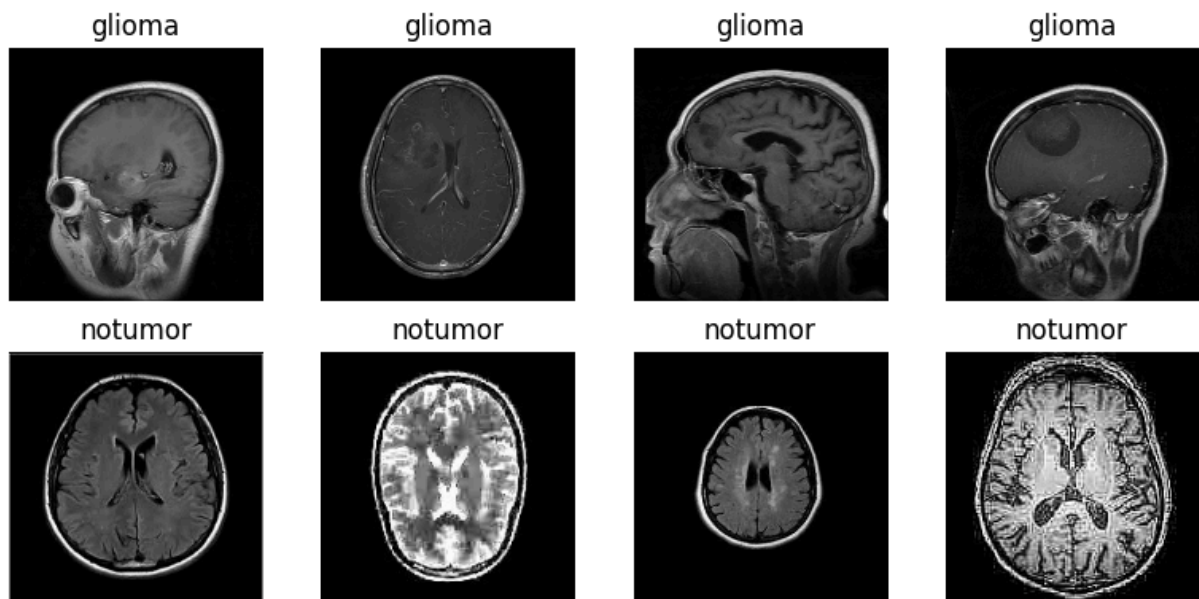
In [6]: *# Quick visual sanity check: Show 4 preprocessed samples per class from the*
class_names = CLASS_NAMES

```
# collect a small batch
xb, yb = next(iter(ds_train))
xb = xb.numpy(); yb = yb.numpy()

fig, axes = plt.subplots(2, 4, figsize=(8,4))
axes = axes.reshape(-1)
shown = {i:0 for i in range(len(class_names))}
for i in range(len(xb)):
    c = int(yb[i])
    if shown[c] >= 4:
        continue
    axes[c*4 + shown[c]].imshow(xb[i,...,0], cmap="gray")
    axes[c*4 + shown[c]].axis("off")
    axes[c*4 + shown[c]].set_title(class_names[c])
    shown[c] += 1
    if all(v==4 for v in shown.values()):
        break
plt.tight_layout(); plt.show()
```

/tmp/ipython-input-1886849759.py:12: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

```
c = int(yb[i])
```



In [7]: *"""*
You'll learn two new layers in this assignment. MCSpatialDropout2D is a smart
networks. Instead of randomly turning off individual pixels, it turns off en
once. This works better for images because it prevents the network from gett

detectors working together, which helps it generalize better to new data. By keeping internal signals stable during training. It takes the outputs from each layer (Batch Normalization data), then applies two learnable parameters to scale and shift the results. This often lets you use higher learning rates, leading to faster and more reliable training.

```

# Keras model factory: Build a baseline CNN
# Force-dropout layers that stay active at inference for MC sampling (Batch Normalization)
@keras.saving.register_keras_serializable(package="mc")
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=None):
        # Force sampling masks even at inference for MC passes
        return super().call(inputs, training=True)

@keras.saving.register_keras_serializable(package="mc")
class MCSpatialDropout2D(tf.keras.layers.SpatialDropout2D):
    def call(self, inputs, training=None):
        # Force sampling masks even at inference for MC passes
        return super().call(inputs, training=True)

def build_cnn(
    dilated=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 1),
    num_classes=1,  # 1 sigmoid unit (binary)
    drop_rate=0.20,
    use_mc_dropout=True,  # if True, use MC* layers; else standard Dropout
    use_spatial=True  # SpatialDropout2D is usually better for convolutional layers
):
    """
    Tiny CNN; toggle dilation in block-2 convs. Adds dropout so MC sampling
    - dilated=False: Conv3x3 (rate=1)
    - dilated=True : dilation rates (2,3,5) in the 2nd conv of each block
    - Dropout: after each block; optionally before Dense head
    """
    rates = (1,1,1)

    Drop = (MCSpatialDropout2D if (use_mc_dropout and use_spatial) else
            L.SpatialDropout2D if use_spatial else
            MCDropout if use_mc_dropout else
            L.Dropout)

    x_in = L.Input(shape=input_shape)
    x = x_in

    # Block A
    x = L.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = L.BatchNormalization()(x)
    x = L.Conv2D(32, 3, padding="same", activation="relu", dilation_rate=rates[0])(x)
    x = L.BatchNormalization()(x)
    x = L.MaxPool2D(pool_size=2)(x)
    x = Drop(drop_rate)(x)  # <-- dropout for MC

    # Block B
    x = L.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = L.BatchNormalization()(x)
    x = L.Conv2D(64, 3, padding="same", activation="relu", dilation_rate=rates[1])(x)
    x = L.BatchNormalization()(x)

```

```

x = L.MaxPool2D(pool_size=2)(x)
x = Drop(drop_rate)(x) # <-- dropout for MC

# Block C
x = L.Conv2D(128, 3, padding="same", activation="relu")(x)
x = L.BatchNormalization()(x)
x = L.Conv2D(128, 3, padding="same", activation="relu", dilation_rate=ra
x = L.BatchNormalization()(x)
x = Drop(drop_rate)(x) # <-- dropout for MC

x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = L.Dropout(drop_rate)(x) # a small dense-level dropout is fine (star

out = tf.keras.layers.Dense(1, activation="sigmoid")(x) # binary head
model = tf.keras.Model(x_in, out)
return model

# Rebuild models (students then train as in Step 14)
model_base = build_cnn(dilated=False)
model_base.summary()

```

Model: "functional"

Layer (type)	Output Shape	Par
input_layer (InputLayer)	(None, 160, 160, 1)	
conv2d (Conv2D)	(None, 160, 160, 32)	
batch_normalization (BatchNormalization)	(None, 160, 160, 32)	
conv2d_1 (Conv2D)	(None, 160, 160, 32)	9
batch_normalization_1 (BatchNormalization)	(None, 160, 160, 32)	
max_pooling2d (MaxPooling2D)	(None, 80, 80, 32)	
mc_spatial_dropout2d (MCSpatialDropout2D)	(None, 80, 80, 32)	
conv2d_2 (Conv2D)	(None, 80, 80, 64)	18
batch_normalization_2 (BatchNormalization)	(None, 80, 80, 64)	
conv2d_3 (Conv2D)	(None, 80, 80, 64)	36
batch_normalization_3 (BatchNormalization)	(None, 80, 80, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 40, 40, 64)	
mc_spatial_dropout2d_1 (MCSpatialDropout2D)	(None, 40, 40, 64)	
conv2d_4 (Conv2D)	(None, 40, 40, 128)	73
batch_normalization_4 (BatchNormalization)	(None, 40, 40, 128)	
conv2d_5 (Conv2D)	(None, 40, 40, 128)	147
batch_normalization_5 (BatchNormalization)	(None, 40, 40, 128)	
mc_spatial_dropout2d_2 (MCSpatialDropout2D)	(None, 40, 40, 128)	
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense (Dense)	(None, 1)	

Total params: 288,353 (1.10 MB)

Trainable params: 287,457 (1.10 MB)

Non-trainable params: 896 (3.50 KB)

Problem 1.1: Explain the CNN

The cell above shows layer of the CNN, with the output tensor shape and the number of trainable parameters. For each layer, explain in a couple of sentences (1) what the purpose of the layer is (2) why the output shape and number of parameters are as shown.

My Response

Overall, we use six convolution layers two at a time, with normalization and pooling between each block. The code is set up so that we can implement dilation for one convolution layer in each block (although for the time being dilation is inactive by setting its parameter to 1). Below, I describe each layer and why the output shape and number of parameters are as shown.

Input: This is our raw image (160 x 160 x 1).

The convolutional layers are how we extract features. Here, we use `relu()`, which are useful for detecting edges. Since we use padding and dilation is off here, the number of parameters will be the same as the input.

- `conv2d`: input is the raw image, so the output is (160 x 160 x 32), where 32 is the number of filters. We fit (3 x 3) grids with padding, so we pick up $(3 \times 3 \times 1) \times 32 + 32 = 320$ parameters.
- `batch_normalization`: After looking up batch normalization, it seems like this is just to keep things stable, and there are 4 parameters per channel, so we get $32 \times 4 = 128$ new parameters.
- `conv2d_1`: input is the output of the last `conv2d`, so now we keep the output shape (160 x 160 x 32) and get $(3 \times 3 \times 32) \times 32 + 32 = 9,248$ parameters.
- `batch_normalization_1`: same as above (4 x 32) parameters
- `max_pooling2d`: Now we pool, keeping the maximum value in the smaller pooling matrices. We learn no new parameters here, but the shape of these small matrices determines our output shape. Here, we move to (80 x 80 x 32).
- `mc_spatial_dropout2d`: Now, we randomly drop feature channels to help prevent overfitting. Again, we need no new parameters, and we keep the same shape.
- `conv2d_2`: Same as above. Params = $(3 \times 3 \times 32) \times 64 + 64 = 18,496$.
- `batch_normalization_2`: Same as above. Params = $4 \times 64 = 256$.
- `conv2d_3`: Same as above. Params = $(3 \times 3 \times 64) \times 64 + 64 = 32,928$.
- `batch_normalization_3`: Same as above. Params = $4 \times 64 = 256$.
- `max_pooling2d_1`: Same as above. No new params. Shape = (40 x 40 x 64).
- `mc_spatial_dropout2d_1`: Same as above. No new params. Shape = (40 x 40 x 64).
- `conv2d_4`: Same as above, now with 128 filters. Params = $(3 \times 3 \times 64) \times 128 + 128 = 73,856$.

- batch_normalization_4: Same as above. Params = $4 \times 128 = 512$.
- conv2d_5: Same as above. Params = $(3 \times 3 \times 128) \times 128 + 128 = 147,584$.
- batch_normalization_5: Same as above. Params = $4 \times 128 = 512$.
- mc_spatial_dropout2d_2: Same as above. No new params. Shape = $(40 \times 40 \times 128)$.
- global_average_pooling2d: More pooling, no new params. We average each channel over the 40×40 map, collapsing to a 128 vector.
- dropout: again, we randomly zero out some features.
- dense: Our final dense layer for a binary classifier using the sigmoid function. We need to train one weight for each feature plus a constant, so $128 + 1 = 129$ parameters.

The following cell now trains the baseline CNN, and evaluates it on test data.

```
In [8]: # Train the baseline model (with validation) + evaluate on TEST + plots

EPOCHS = 10
OUTDIR = "/content/outputs_cnn_dilation"
os.makedirs(OUTDIR, exist_ok=True)

def compile_model(m):
    m.compile(optimizer=tf.keras.optimizers.Adam(3e-5),
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
              metrics=[tf.keras.metrics.BinaryAccuracy(name="accuracy", threshold=0.5)])
    return m

def train_model(model, name, train_ds, val_ds, epochs=EPOCHS):
    ckpt = os.path.join(OUTDIR, f"{name}.keras") # best checkpoint
    cbs = [ModelCheckpoint(ckpt, monitor="val_accuracy", save_best_only=True)]
    t0 = time.time()
    hist = model.fit(train_ds, validation_data=val_ds, epochs=epochs, callbacks=cbs)
    train_seconds = time.time() - t0
    print(f"[TIMER] {name}: training wall-clock = {train_seconds:.2f} sec "
          f"({train_seconds/max(1, len(hist.history['loss'])):.2f} sec/epoch)")
    # OPTIONAL: save the final weights separately (do NOT overwrite ckpt)
    final_path = os.path.join(OUTDIR, f"{name}_final.keras")
    model.save(final_path)
    return ckpt, hist, train_seconds

def evaluate_and_report(model, ds, class_names):
    y_true, y_pred = [], []
    out_last_dim = model.output_shape[-1] # 1 for sigmoid, 2 for softmax
    for xb, yb in ds:
        probs = model.predict(xb, verbose=0)
        probs = np.array(probs).reshape((probs.shape[0], -1))
        preds = (probs[:,0] >= 0.5).astype(int) if out_last_dim == 1 else np.argmax(probs, axis=-1)
        yt = yb.numpy().reshape(-1).astype(int)
        y_true.append(yt); y_pred.append(preds)
    y_true = np.concatenate(y_true); y_pred = np.concatenate(y_pred)
    print(classification_report(y_true, y_pred, target_names=class_names, digits=2))
    print("Confusion matrix:\n", confusion_matrix(y_true, y_pred))
    return y_true, y_pred
```

```

def measure_eval_time(model, ds, n_samples):
    t0 = time.time()
    loss, acc = model.evaluate(ds, verbose=0)
    eval_seconds = time.time() - t0
    print(f"[TIMER] evaluate: {eval_seconds:.2f} sec")
    t1 = time.time()
    _ = model.predict(ds, verbose=0)
    pred_seconds = time.time() - t1
    ips = n_samples / pred_seconds if pred_seconds > 0 else float("inf")
    print(f"[TIMER] predict : {pred_seconds:.2f} sec (~{ips:.1f} images/sec)")
    return loss, acc, eval_seconds, pred_seconds, ips

# --- compile
model_base = compile_model(model_base)

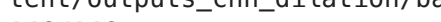
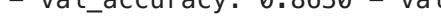
# --- train with validation (no test leakage)
ckpt_base, hist_base, train_time_base = train_model(model_base, "baseline_cr")

# --- reload best and evaluate on TEST (timed)
model_base = tf.keras.models.load_model(ckpt_base)

print("\n[TEST] Baseline:")
y_true_base, y_pred_base = evaluate_and_report(model_base, ds_test, CLASS_NAMES)
loss_b, acc_b, eval_t_b, pred_t_b, ips_b = measure_eval_time(model_base, ds_test, n_samples)

print(f"\n[TEST] Baseline -> loss: {loss_b:.4f} | acc: {acc_b:.4f}")

```

Epoch 1/10
146/146  **0s** 1s/step - accuracy: 0.6114 - loss: 0.6544
Epoch 1: val_accuracy improved from -inf to 0.45377, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **286s** 2s/step - accuracy: 0.6118 - loss: 0.6541
- val_accuracy: 0.4538 - val_loss: 1.0676
Epoch 2/10
146/146  **0s** 47ms/step - accuracy: 0.7121 - loss: 0.5585
Epoch 2: val_accuracy did not improve from 0.45377
146/146  **10s** 65ms/step - accuracy: 0.7122 - loss: 0.5583
- val_accuracy: 0.4538 - val_loss: 0.9833
Epoch 3/10
145/146  **0s** 56ms/step - accuracy: 0.7589 - loss: 0.5087
Epoch 3: val_accuracy improved from 0.45377 to 0.53425, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **10s** 67ms/step - accuracy: 0.7592 - loss: 0.5083
- val_accuracy: 0.5342 - val_loss: 0.7765
Epoch 4/10
145/146  **0s** 57ms/step - accuracy: 0.8126 - loss: 0.4391
Epoch 4: val_accuracy improved from 0.53425 to 0.76541, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **10s** 69ms/step - accuracy: 0.8127 - loss: 0.4390
- val_accuracy: 0.7654 - val_loss: 0.4827
Epoch 5/10
146/146  **0s** 47ms/step - accuracy: 0.8230 - loss: 0.4032
Epoch 5: val_accuracy improved from 0.76541 to 0.86301, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **9s** 60ms/step - accuracy: 0.8230 - loss: 0.4032
- val_accuracy: 0.8630 - val_loss: 0.3587
Epoch 6/10
145/146  **0s** 54ms/step - accuracy: 0.8556 - loss: 0.3531
Epoch 6: val_accuracy did not improve from 0.86301
146/146  **10s** 65ms/step - accuracy: 0.8555 - loss: 0.3531
- val_accuracy: 0.8596 - val_loss: 0.3363
Epoch 7/10
146/146  **0s** 56ms/step - accuracy: 0.8615 - loss: 0.3408
Epoch 7: val_accuracy improved from 0.86301 to 0.88356, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **10s** 67ms/step - accuracy: 0.8615 - loss: 0.3408
- val_accuracy: 0.8836 - val_loss: 0.2911
Epoch 8/10
145/146  **0s** 59ms/step - accuracy: 0.8650 - loss: 0.3154
Epoch 8: val_accuracy did not improve from 0.88356
146/146  **11s** 75ms/step - accuracy: 0.8651 - loss: 0.3153
- val_accuracy: 0.8784 - val_loss: 0.3116
Epoch 9/10
146/146  **0s** 46ms/step - accuracy: 0.8846 - loss: 0.2958
Epoch 9: val_accuracy improved from 0.88356 to 0.90582, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **9s** 58ms/step - accuracy: 0.8846 - loss: 0.2957
- val_accuracy: 0.9058 - val_loss: 0.2511
Epoch 10/10
145/146  **0s** 56ms/step - accuracy: 0.8883 - loss: 0.2733
Epoch 10: val_accuracy improved from 0.90582 to 0.91610, saving model to /content/outputs_cnn_dilation/baseline_cnn.keras
146/146  **10s** 67ms/step - accuracy: 0.8885 - loss: 0.2732


```
- val_accuracy: 0.9161 - val_loss: 0.2357  
[TIMER] baseline_cnn: training wall-clock = 373.61 sec (37.36 sec/epoch) (monitoring val_accuracy)
```

[TEST] Baseline:

	precision	recall	f1-score	support
glioma	0.8251	0.9433	0.8802	300
notumor	0.9530	0.8519	0.8996	405
accuracy			0.8908	705
macro avg	0.8891	0.8976	0.8899	705
weighted avg	0.8986	0.8908	0.8914	705

Confusion matrix:

```
[[283  17]  
 [ 60 345]]
```

[TIMER] evaluate: 5.03 sec

[TIMER] predict : 6.32 sec (~111.5 images/sec)

[TEST] Baseline -> loss: 0.2714 | acc: 0.8908

Problem 1.2: Improve on the baseline with a dilated CNN

Your job is now to use the above code, appropriately modified as needed, to build and train a dilated CNN that has better accuracy than the baseline.

Note that the keras Conv2D method can handle dilation. You may experiment with any architecture (combination of layers) and dilation parameters that you wish.

To keep the notebook clear, only show your best model, not intermediate, experimental models. Explain your choice of model, and why your best dilation CNN model does or does not improve on the baseline. Note that you should use a runtime environment that uses a GPU for faster processing speed.

I only messed around a bit with the dilation, and I didn't really change the structure otherwise. It seems like the dilation dimensions suggested in the comments of the code provided do work really well. We managed to improve our accuracy from about 84% to almost 94% (especially improving precision of glioma and recall of notumor). It seems like there is some important spatial information linked across a large area. Dilation allows us to increase the area of our filters without getting too much more computational cost.

```
In [9]: # My final CNN (building off the baseline structure):  
  
# Force-dropout layers that stay active at inference for MC sampling (Batch Normalization)  
@keras.saving.register_keras_serializable(package="mc")  
class MCDropout(tf.keras.layers.Dropout):  
    def call(self, inputs, training=None):  
        # Force sampling masks even at inference for MC passes  
        return super().call(inputs, training=True)
```

```

@keras.saving.register_keras_serializable(package="mc")
class MCSpatialDropout2D(tf.keras.layers.SpatialDropout2D):
    def call(self, inputs, training=None):
        # Force sampling masks even at inference for MC passes
        return super().call(inputs, training=True)

def build_cnn(
    dilated=True,
    input_shape=(IMG_SIZE, IMG_SIZE, 1),
    num_classes=1,                # 1 sigmoid unit (binary)
    drop_rate=0.20,
    use_mc_dropout=True,          # if True, use MC* layers; else standard D
    use_spatial=True             # SpatialDropout2D is usually better for c
):
    """
    Tiny CNN; toggle dilation in block-2 convs. Adds dropout so MC sampling
    - dilated=False: Conv3x3 (rate=1)
    - dilated=True : dilation rates (2,3,5) in the 2nd conv of each block
    - Dropout: after each block; optionally before Dense head
    """
    rates = (2,3,5)

    Drop = (MCSpatialDropout2D if (use_mc_dropout and use_spatial) else
            L.SpatialDropout2D if use_spatial else
            MCDropout if use_mc_dropout else
            L.Dropout)

    x_in = L.Input(shape=input_shape)
    x = x_in

    # Block A
    x = L.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = L.BatchNormalization()(x)
    x = L.Conv2D(32, 3, padding="same", activation="relu", dilation_rate=rates[0])(x)
    x = L.BatchNormalization()(x)
    x = L.MaxPool2D(pool_size=2)(x)
    x = Drop(drop_rate)(x) # <-- dropout for MC

    # Block B
    x = L.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = L.BatchNormalization()(x)
    x = L.Conv2D(64, 3, padding="same", activation="relu", dilation_rate=rates[1])(x)
    x = L.BatchNormalization()(x)
    x = L.MaxPool2D(pool_size=2)(x)
    x = Drop(drop_rate)(x) # <-- dropout for MC

    # Block C
    x = L.Conv2D(128, 3, padding="same", activation="relu")(x)
    x = L.BatchNormalization()(x)
    x = L.Conv2D(128, 3, padding="same", activation="relu", dilation_rate=rates[2])(x)
    x = L.BatchNormalization()(x)
    x = Drop(drop_rate)(x) # <-- dropout for MC

    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = L.Dropout(drop_rate)(x) # a small dense-level dropout is fine (star

```

```
    out = tf.keras.layers.Dense(1, activation="sigmoid")(x) # binary head
    model = tf.keras.Model(x_in, out)
    return model

# Rebuild models (students then train as in Step 14)
model_final = build_cnn(dilated=True)
model_final.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Par
input_layer_1 (InputLayer)	(None, 160, 160, 1)	
conv2d_6 (Conv2D)	(None, 160, 160, 32)	
batch_normalization_6 (BatchNormalization)	(None, 160, 160, 32)	
conv2d_7 (Conv2D)	(None, 160, 160, 32)	9
batch_normalization_7 (BatchNormalization)	(None, 160, 160, 32)	
max_pooling2d_2 (MaxPooling2D)	(None, 80, 80, 32)	
mc_spatial_dropout2d_3 (MCSpatialDropout2D)	(None, 80, 80, 32)	
conv2d_8 (Conv2D)	(None, 80, 80, 64)	18
batch_normalization_8 (BatchNormalization)	(None, 80, 80, 64)	
conv2d_9 (Conv2D)	(None, 80, 80, 64)	36
batch_normalization_9 (BatchNormalization)	(None, 80, 80, 64)	
max_pooling2d_3 (MaxPooling2D)	(None, 40, 40, 64)	
mc_spatial_dropout2d_4 (MCSpatialDropout2D)	(None, 40, 40, 64)	
conv2d_10 (Conv2D)	(None, 40, 40, 128)	73
batch_normalization_10 (BatchNormalization)	(None, 40, 40, 128)	
conv2d_11 (Conv2D)	(None, 40, 40, 128)	147
batch_normalization_11 (BatchNormalization)	(None, 40, 40, 128)	
mc_spatial_dropout2d_5 (MCSpatialDropout2D)	(None, 40, 40, 128)	
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	
dropout_1 (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 1)	

Total params: 288,353 (1.10 MB)

Trainable params: 287,457 (1.10 MB)

Non-trainable params: 896 (3.50 KB)

In [10]: *# Train the final model (with validation) + evaluate on TEST + plots*

```
EPOCHS = 10
OUTDIR = "/content/outputs_cnn_dilation"
os.makedirs(OUTDIR, exist_ok=True)

def compile_model(m):
    m.compile(optimizer=tf.keras.optimizers.Adam(3e-5),
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
              metrics=[tf.keras.metrics.BinaryAccuracy(name="accuracy", threshold=0.5)])
    return m

def train_model(model, name, train_ds, val_ds, epochs=EPOCHS):
    ckpt = os.path.join(OUTDIR, f"{name}.keras") # best checkpoint
    cbs = [ModelCheckpoint(ckpt, monitor="val_accuracy", save_best_only=True)]
    t0 = time.time()
    hist = model.fit(train_ds, validation_data=val_ds, epochs=epochs, callbacks=cbs)
    train_seconds = time.time() - t0
    print(f"[TIMER] {name}: training wall-clock = {train_seconds:.2f} sec "
          f"({train_seconds/max(1, len(hist.history['loss'])):.2f} sec/epoch)")
    # OPTIONAL: save the final weights separately (do NOT overwrite ckpt)
    final_path = os.path.join(OUTDIR, f"{name}_final.keras")
    model.save(final_path)
    return ckpt, hist, train_seconds

def evaluate_and_report(model, ds, class_names):
    y_true, y_pred = [], []
    out_last_dim = model.output_shape[-1] # 1 for sigmoid, 2 for softmax
    for xb, yb in ds:
        probs = model.predict(xb, verbose=0)
        probs = np.array(probs).reshape((probs.shape[0], -1))
        preds = (probs[:,0] >= 0.5).astype(int) if out_last_dim == 1 else np.argmax(probs, axis=-1).astype(int)
        yt = yb.numpy().reshape(-1).astype(int)
        y_true.append(yt); y_pred.append(preds)
    y_true = np.concatenate(y_true); y_pred = np.concatenate(y_pred)
    print(classification_report(y_true, y_pred, target_names=class_names, digits=2))
    print("Confusion matrix:\n", confusion_matrix(y_true, y_pred))
    return y_true, y_pred

def measure_eval_time(model, ds, n_samples):
    t0 = time.time()
    loss, acc = model.evaluate(ds, verbose=0)
    eval_seconds = time.time() - t0
    print(f"[TIMER] evaluate: {eval_seconds:.2f} sec")
    t1 = time.time()
    _ = model.predict(ds, verbose=0)
    pred_seconds = time.time() - t1
    ips = n_samples / pred_seconds if pred_seconds > 0 else float("inf")
    print(f"[TIMER] predict : {pred_seconds:.2f} sec (~{ips:.1f} images/sec)")
    return loss, acc, eval_seconds, pred_seconds, ips

# --- compile
model_final = compile_model(model_final)

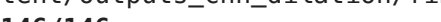
# --- train with validation (no test leakage)
```

```
ckpt_base, hist_base, train_time_base = train_model(model_final, "final_cnn")

# --- reload best and evaluate on TEST (timed)
model_final = tf.keras.models.load_model(ckpt_base)

print("\n[TEST] Final:")
y_true_base, y_pred_base = evaluate_and_report(model_final, ds_test, CLASS_M
loss_b, acc_b, eval_t_b, pred_t_b, ips_b = measure_eval_time(model_final, ds

print(f"\n[TEST] Final -> loss: {loss_b:.4f} | acc: {acc_b:.4f}")
```

Epoch 1/10
146/146  **0s** 102ms/step - accuracy: 0.6278 - loss: 0.6373
Epoch 1: val_accuracy improved from -inf to 0.45377, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **33s** 143ms/step - accuracy: 0.6281 - loss: 0.6371 - val_accuracy: 0.4538 - val_loss: 0.9108
Epoch 2/10
145/146  **0s** 57ms/step - accuracy: 0.7468 - loss: 0.5194
Epoch 2: val_accuracy improved from 0.45377 to 0.47603, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **10s** 69ms/step - accuracy: 0.7470 - loss: 0.5191 - val_accuracy: 0.4760 - val_loss: 0.8969
Epoch 3/10
146/146  **0s** 57ms/step - accuracy: 0.8160 - loss: 0.4338
Epoch 3: val_accuracy improved from 0.47603 to 0.65925, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **11s** 76ms/step - accuracy: 0.8161 - loss: 0.4337 - val_accuracy: 0.6592 - val_loss: 0.6127
Epoch 4/10
145/146  **0s** 51ms/step - accuracy: 0.8447 - loss: 0.3743
Epoch 4: val_accuracy improved from 0.65925 to 0.77568, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **10s** 68ms/step - accuracy: 0.8449 - loss: 0.3740 - val_accuracy: 0.7757 - val_loss: 0.4382
Epoch 5/10
146/146  **0s** 48ms/step - accuracy: 0.8890 - loss: 0.3089
Epoch 5: val_accuracy improved from 0.77568 to 0.85959, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **9s** 59ms/step - accuracy: 0.8890 - loss: 0.3088 - val_accuracy: 0.8596 - val_loss: 0.3415
Epoch 6/10
145/146  **0s** 57ms/step - accuracy: 0.9115 - loss: 0.2596
Epoch 6: val_accuracy improved from 0.85959 to 0.88870, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **10s** 69ms/step - accuracy: 0.9115 - loss: 0.2597 - val_accuracy: 0.8887 - val_loss: 0.3000
Epoch 7/10
146/146  **0s** 64ms/step - accuracy: 0.9240 - loss: 0.2365
Epoch 7: val_accuracy improved from 0.88870 to 0.91952, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **11s** 75ms/step - accuracy: 0.9240 - loss: 0.2365 - val_accuracy: 0.9195 - val_loss: 0.2284
Epoch 8/10
145/146  **0s** 48ms/step - accuracy: 0.9191 - loss: 0.2208
Epoch 8: val_accuracy did not improve from 0.91952
146/146  **9s** 61ms/step - accuracy: 0.9191 - loss: 0.2208 - val_accuracy: 0.9144 - val_loss: 0.2278
Epoch 9/10
146/146  **0s** 54ms/step - accuracy: 0.9200 - loss: 0.2213
Epoch 9: val_accuracy improved from 0.91952 to 0.92466, saving model to /content/outputs_cnn_dilation/final_cnn.keras
146/146  **10s** 66ms/step - accuracy: 0.9200 - loss: 0.2213 - val_accuracy: 0.9247 - val_loss: 0.2006
Epoch 10/10
145/146  **0s** 56ms/step - accuracy: 0.9302 - loss: 0.2066
Epoch 10: val_accuracy improved from 0.92466 to 0.93836, saving model to /co

```

ntent/outputs_cnn_dilation/final_cnn.keras
146/146 ————— 10s 67ms/step - accuracy: 0.9303 - loss: 0.2064
- val_accuracy: 0.9384 - val_loss: 0.1815
[TIMER] final_cnn: training wall-clock = 123.36 sec (12.34 sec/epoch) (monitoring val_accuracy)

```

```

[TEST] Final:
              precision    recall  f1-score   support

   glioma      0.8938      0.9533      0.9226        300
  notumor      0.9636      0.9160      0.9392        405

 accuracy              0.9319        705
 macro avg      0.9287      0.9347      0.9309        705
weighted avg      0.9339      0.9319      0.9322        705

```

```

Confusion matrix:
[[286  14]
 [ 34 371]]
[TIMER] evaluate: 5.31 sec
[TIMER] predict : 5.62 sec (~125.5 images/sec)

```

```

[TEST] Final -> loss: 0.1623 | acc: 0.9433

```

Problem 1.3 Monte Carlo dropout to assess model uncertainty

Your CNN should use Monte Carlo (MC) dropout to compute the risk at test time. For a fixed MRI image scan x , you obtain T stochastic predictions $\{\hat{y}_t\}_{t=1}^T$.

(a) Mean and variance from MC passes. Give mathematical expressions for the MC estimate of the predictive mean μ_T and its variance s_T^2 from $\{\hat{y}_t\}$.

[your markdown or scanned solution here]

(b) Accuracy guarantee. Using Chebyshev's inequality, derive a lower bound on T ensuring $\Pr(|\mu_T - \mu_{\text{true}}| \geq \varepsilon) \leq \delta$. Express the minimal T in terms of $\text{Var}(\hat{y}_t)$ (or a plug-in estimate). Clearly state any assumptions you need to make, such as Gaussianity.

[your markdown or scanned solution here]

(c) Numerical budget. With target $\varepsilon = 0.05$, confidence $1 - \delta = 0.95$ ($\delta = 0.05$), a pilot of $T_0 = 50$ passes giving standard deviation $s_{T_0}^2 = 0.08$, compute the required T from part (b). Describe what this means--what guarantee does this give about the accuracy of the prediction?

[your markdown or scanned solution here]

Add blockquote

1.3

$$a) \quad \mu_T = T^{-1} \sum_{t=1}^T \hat{y}_t$$

$$S_T^2 = (T-1)^{-1} \sum_{t=1}^T (\hat{y}_t - \mu_T)^2$$

b) With Chebyshev, we don't need to make distribution assumptions (although these would allow us to make tighter bounds)

$$P(|\mu_T - \mu_0| > \epsilon) \leq \frac{\text{Var}(\mu_T)}{\epsilon^2} = \frac{\sigma^2}{T\epsilon^2}, \text{ where } \sigma^2 = \text{Var}(\hat{y}_t)$$

$$\text{For } \delta > 0, \text{ we see that } T \geq \frac{\sigma^2}{\delta\epsilon^2}$$

$$c) \quad T \geq \frac{0.08}{(0.05)(0.05)^2} = 640$$

So, if we run at least 640 MC dropout passes,

$|\mu_T - \mu_0| < 0.05$ with 95% confidence level.

[Here, μ_0 is the true predictive mean.]

Problem 1.4 Monte Carlo (MC) Dropout on test images

You will implement MC Dropout to quantify predictive uncertainty at test time. By keeping Dropout active during inference and sampling the model multiple times on the same inputs, you will approximate the posterior predictive distribution and summarize it with a mean prediction and a dispersion measure (standard deviation). Use MC Dropout to approximate the posterior predictive and quantify epistemic uncertainty at test time. Keep dropout on during inference, sample the model multiple times on the same inputs, and summarize the resulting distribution. Set the number of stochastic forward passes to a reasonable value based on the outputs you obtain. Remember by default `model.predict` does not keep dropout active at inference.

In this part you will:

1. Build A sampler function that deterministically picks the first 5 images per class from the test set folder for each label (10 images in total)
2. A stochastic inference predictive route that runs the model with dropout active and returns one probability per image in each round. Your CNN models is preferably defined as a function so that you can feed the baseline and dilated CNNs to compare the values here.
3. Calculate the mean and standard deviation of the stochastic forward passes in step 2 to generate a table or grid that depicts indices, y_true, y_hat, p_mean, p_std. The two latter values should be obtained with your stochastic forward passes with dropout on.
4. See if there is a mispredicted label based on your y_hat, then discuss the correlation between standard deviation of the stochastic runs and the final y_hat.

N.B. Here, I used GPT pretty heavily to get the code compact and nice-looking.

```
In [11]: import pandas as pd
from scipy.stats import pearsonr

# ---- Parameters ----
MC_PASSES = 100 # number of stochastic forward passes
N_PER_CLASS = 5 # number of test images per class to sample

# ---- 1. Deterministic sampler ----
def sample_test_images(paths, labels, class_names, n_per_class=N_PER_CLASS):
    sel_paths, sel_labels = [], []
    for c in range(len(class_names)):
        # pick first n_per_class sorted paths for each class
        idxs = np.where(labels == c)[0]
        chosen = sorted(paths[idxs])[:n_per_class]
        sel_paths.extend(chosen)
        sel_labels.extend([c]*len(chosen))
    return np.array(sel_paths), np.array(sel_labels)

sel_paths, sel_labels = sample_test_images(test_paths, test_labels, CLASS_NAMES)
ds_mc = make_dataset(sel_paths, sel_labels, batch_size=len(sel_paths), shuffle=True)

# ---- 2. Stochastic inference with dropout ----
xb, yb = next(iter(ds_mc)) # single batch of size = 10
y_true = yb.numpy().reshape(-1).astype(int)

mc_preds = []
for t in range(MC_PASSES):
    probs = model_final.predict(xb, verbose=0) # MC dropout active by design
    mc_preds.append(probs.reshape(-1))
mc_preds = np.stack(mc_preds, axis=0) # shape (T, N)

# ---- 3. Mean and std across passes ----
p_mean = mc_preds.mean(axis=0)
p_std = mc_preds.std(axis=0, ddof=1)
```

```

y_hat = (p_mean >= 0.5).astype(int)

# ---- 4. Results table ----
results = pd.DataFrame({
    "idx": np.arange(len(y_true)),
    "class": [CLASS_NAMES[c] for c in y_true],
    "y_true": y_true,
    "y_hat": y_hat,
    "p_mean": p_mean,
    "p_std": p_std
})

print(results)

# ---- 5. Correlation between std and errors ----
abs_error = np.abs(p_mean - y_true)
error_indicator = (y_hat != y_true).astype(int)

corr_abs, _ = pearsonr(p_std, abs_error)
corr_err, _ = pearsonr(p_std, error_indicator)

print(f"\nCorrelation between p_std and |error|: {corr_abs:.3f}")
print(f"Correlation between p_std and misclassification indicator: {corr_err:.3f}")

```

	idx	class	y_true	y_hat	p_mean	p_std
0	0	glioma	0	0	0.234888	0.143043
1	1	glioma	0	0	0.176229	0.108741
2	2	glioma	0	0	0.029214	0.043460
3	3	glioma	0	0	0.124281	0.100169
4	4	glioma	0	0	0.172867	0.112283
5	5	notumor	1	0	0.354688	0.169722
6	6	notumor	1	1	0.990784	0.013451
7	7	notumor	1	1	0.941953	0.084369
8	8	notumor	1	1	0.581239	0.202155
9	9	notumor	1	1	0.726463	0.189216

Correlation between p_std and |error|: 0.810

Correlation between p_std and misclassification indicator: 0.306

First, running this on both the baseline and final versions of the model shows that the final version indeed has much less uncertainty in the test set (I don't show the baseline results here to save space, but the differences were quite clear).

We see that we only make one incorrect prediction in this test of our final model, and that for that case the mean is much closer to the 0.5 cutoff with one of the higher variances. For the others, we get quite a few good cases where the mean predicted probability is quite high or low and the variance is relatively low. Overall, it seems like we've done a nice job :)

Problem 2: All that glitters (20 points)

In this problem you will use Gaussian process regression to model the trends in gold medal performances of selected events in the summer Olympics. The objectives of this problem are for you to:

- Gain experience with Gaussian processes, to better understand how they work
- Explore how posterior inference depends on the properties of the prior mean and kernel
- Use Bayesian inference to identify unusual events
- Practice making your Python code modular and reusable

For this problem, the only starter code we provide is to read in the data and extract one event. You may write any GP code that you choose to, but please do not use any package for Gaussian processes; your code should be "np-complete" (using only basic `numpy` methods). You are encouraged to start from the [GP demo code](#) used in class.

When we ran the GP demo code from class on the marathon data, it generated the following plot: No description has been provided for this image

Note several properties of this plot:

- It shows the Bayesian confidence of the regression, as a shaded area. This is a 95% confidence band because it has width given by $\pm 2\sqrt{V}$, where V is the estimated variance. The variance increases at the right side, for future years.
- The gold medal time for the 1904 marathon is outside of this confidence band. In fact, the 1904 marathon was an [unusual event](#), and this is apparent from the model.
- The plot shows the posterior mean, and also shows one random sample from the posterior distribution.

Your task in this problem is generate such a plot for six different Olympic events by writing a function

```
def gp_olympic_event(year, result, kernel, mean, noise,
event_name):    ...
```

where the input variables are the following:

- `year` : a numpy array of years (integers)
- `result` : a numpy array of numerical results, for the gold medal performances in that event
- `kernel` : a kernel function
- `mean` : a mean function
- `noise` : a single float for the variance of the noise, σ^2
- `event_name` : a string used to label the y-axis, for example "marathon min/mile (men's event)"

Your function should compute the Gaussian process regression, and then display the resulting plot, analogous to the plot above for the men's marathon event.

You will then process **six** of the events, three men's events and three women's events, and call your function to generate the corresponding six plots.

For each event, you should create a markdown cell that describes the resulting model. Comment on such things as:

- How you chose the kernel, mean, and noise.
- Why the plot does or doesn't look satisfactory to you
- If there are any events such as the 1904 marathon that are notable.
- What happens to the posterior mean (for example during WWII) if there are gaps in the data

Use your best judgement to describe your findings; post questions to EdD if things are unclear. And have fun!

In the remainder of this problem description, we recall how we processed the marathon data, as an example. The following cell reads in the data and displays the collection of events that are included in the dataset.

```
In [12]: import numpy as np
import pandas as pd

dat = pd.read_csv('https://raw.githubusercontent.com/YData123/sds365-fa25/main/marathon_events.csv')
events = set(np.array(dat['Event']))
print(events)

{'100M Men', '400M Women', 'Marathon Men', 'Discus Throw Men', '50Km Race Walk Men', '4X400M Relay Women', 'Hammer Throw Women', '4X100M Relay Men', '1500M Men', 'Discus Throw Women', 'Pole Vault Men', '100M Hurdles Women', '5000M Women', 'Hammer Throw Men', '10000M Men', '110M Hurdles Men', '400M Men', 'Javelin Throw Men', '100M Women', 'Pole Vault Women', 'Triple Jump Men', '20Km Race Walk Men', 'Decathlon Men', '200M Women', '400M Hurdles Men', '200M Men', 'Heptathlon Women', 'High Jump Men', '3000M Steeplechase Men', 'Long Jump Men', 'Shot Put Men', 'Marathon Women', 'Shot Put Women', 'Long Jump Women', '800M Women', '4X400M Relay Men', '3000M Steeplechase Women', '4X100M Relay Women', 'Javelin Throw Women', '400M Hurdles Women', '5000M Men', 'Triple Jump Women', '20Km Race Walk Women', '10000M Women', 'High Jump Women', '1500M Women', '800M Men'}
```

We then process the time to compute the minutes per mile (without checking that the race was actually 26.2 miles!)

```
In [13]: marathon = dat[dat['Event'] == 'Marathon Men']
marathon = marathon[marathon['Medal'] == 'G']
marathon = marathon.sort_values('Year')
time = np.array(marathon['Result'])
```

```

mpm = []
for tm in time:
    t = np.array(tm.split(':'), dtype=float)
    minutes_per_mile = (t[0]*60*60 + t[1]*60 + t[2])/(60*26.2)
    mpm.append(minutes_per_mile)

marathon['Minutes per Mile'] = np.round(mpm,2)
marathon = marathon.drop(columns=['Gender', 'Event'], axis=1)
marathon.reset_index(drop=True, inplace=True)
year = np.array(marathon['Year'])
result = np.array(marathon['Minutes per Mile'])
marathon

# Process women's marathon
wmarathon = dat[dat['Event'] == 'Marathon Women']
wmarathon = wmarathon[wmarathon['Medal']=='G']
wmarathon = wmarathon.sort_values('Year')
time = np.array(wmarathon['Result'])
wmpm = []
for tm in time:
    t = np.array(tm.split(':'), dtype=float)
    minutes_per_mile = (t[0]*60*60 + t[1]*60 + t[2])/(60*26.2)
    wmpm.append(minutes_per_mile)

wmarathon['Minutes per Mile'] = np.round(wmpm,2)
wmarathon = wmarathon.drop(columns=['Gender', 'Event'], axis=1)
wmarathon.reset_index(drop=True, inplace=True)
wyear = np.array(wmarathon['Year'])
result = np.array(wmarathon['Minutes per Mile'])
wmarathon

# Process men's 100M
m100 = dat[dat['Event'] == '100M Men']
m100 = m100[m100['Medal']=='G']
m100 = m100.sort_values('Year')
m100_time = np.array(m100['Result'])

m100 = m100.drop(columns=['Gender', 'Event'], axis=1)
m100.reset_index(drop=True, inplace=True)
m100_year = np.array(m100['Year'])
m100

# Process women's 100M
w100 = dat[dat['Event'] == '100M Women']
w100 = w100[w100['Medal']=='G']
w100 = w100.sort_values('Year')
w100_time = np.array(w100['Result'])

w100 = w100.drop(columns=['Gender', 'Event'], axis=1)
w100.reset_index(drop=True, inplace=True)
w100_year = np.array(w100['Year'])
w100

# Process men's high jump
mhigh = dat[dat['Event'] == 'High Jump Men']
mhigh = mhigh[mhigh['Medal']=='G']

```

```

mhigh = mhigh.sort_values('Year')
mhigh_height = np.array(mhigh['Result'])

mhigh = mhigh.drop(columns=['Gender', 'Event'], axis=1)
mhigh.reset_index(drop=True, inplace=True)
mhigh_year = np.array(mhigh['Year'])
mhigh

# Process women's high jump
whigh = dat[dat['Event'] == 'High Jump Women']
whigh = whigh[whigh['Medal']=='G']
whigh = whigh.sort_values('Year')
whigh = whigh.dropna(subset=["Result"]) # I found that the 2016 result is mi
whigh_height = np.array(whigh['Result'])

whigh = whigh.drop(columns=['Gender', 'Event'], axis=1)
whigh.reset_index(drop=True, inplace=True)
whigh_year = np.array(whigh['Year'])
whigh

```

Out[13]:

	Location	Year	Medal	Name	Nationality	Result
0	Los Angeles	1932	G	Jean SHILEY	USA	1.65
1	Berlin	1936	G	Ibolya CSÁK	HUN	1.6
2	London	1948	G	Alice COACHMAN	USA	1.68
3	Helsinki	1952	G	Esther BRAND	RSA	1.67
4	Rome	1960	G	Iolanda BALAS	ROU	1.85
5	Tokyo	1964	G	Iolanda BALAS	ROU	1.9
6	Montreal	1976	G	Rosemarie WITSCHAS-ACKERMANN	GDR	1.93
7	Moscow	1980	G	Sara SIMEONI	ITA	1.97
8	Barcelona	1992	G	Heike HENKEL	GER	2.02
9	Atlanta	1996	G	Stefka KOSTADINOVA	BUL	2.05
10	Sydney	2000	G	Yelena YELESINA	RUS	2.01
11	Athens	2004	G	Elena SLESARENKO	RUS	2.06
12	Beijing	2008	G	Tia HELLEBAUT	BEL	2.05
13	London	2012	G	Anna CHICHEROVA	RUS	2.05

Enter your code and markdown following this cell.

```

In [14]: import numpy as np
import matplotlib.pyplot as plt

def gp_olympic_event(year, result, kernel, mean, noise, event_name):

```

```

"""
Gaussian process regression for Olympic gold medal results in a single e

year : numpy array of years (integers)
result: numpy array of numerical results (floats)
kernel: function K(x, z) returning covariance matrix
mean : function m(x) returning prior mean
noise : float, variance of noise ( $\sigma^2$ )
event_name : str, label for y-axis
"""

# Training inputs
Xtrain = np.array(year, dtype=float)
ytrain = np.array(result, dtype=float)

# Grid of test inputs (years)
xs = np.linspace(min(year) - 4, max(year) + 8, 400)

# Kernel matrices
K = kernel(Xtrain, Xtrain) + noise * np.eye(len(Xtrain))
Ks = kernel(Xtrain, xs)
Kss = kernel(xs, xs) + noise * np.eye(len(xs))

# Mean functions
mu_train = mean(Xtrain)
mu_test = mean(xs)

# Solve for posterior
Ki = np.linalg.inv(K)
postMu = mu_test + Ks.T @ Ki @ (ytrain - mu_train)
postCov = Kss - Ks.T @ Ki @ Ks
S2 = np.diag(postCov)

# One posterior sample
L = np.linalg.cholesky(postCov + 1e-8 * np.eye(len(xs)))
f_sample = postMu + L @ np.random.randn(len(xs))

# Plot
fig, ax = plt.subplots(figsize=(9,6))
ax.fill_between(xs, postMu - 2*np.sqrt(S2), postMu + 2*np.sqrt(S2),
               alpha=0.2, label='95% confidence band')
ax.plot(xs, postMu, color='black', lw=2, label='posterior mean')
ax.plot(xs, f_sample, color='red', lw=1, alpha=0.7, label='posterior sam
ax.scatter(Xtrain, ytrain, color='blue', marker='o', label='observations
ax.set_xlabel("Year", fontsize=14)
ax.set_ylabel(event_name, fontsize=14)
ax.legend()
plt.show()

```

```

In [15]: # RBF kernel
def rbf_kernel1(x, z, h=7, sigma_f=0.5):
    x = x.reshape(-1,1)
    z = z.reshape(-1,1)
    dists = (x - z.T)**2
    return sigma_f**2 * np.exp(-dists / (2*h**2))

def rbf_kernel2(x, z, h=7, sigma_f=1):

```



```

    x = x.reshape(-1,1)
    z = z.reshape(-1,1)
    dists = (x - z.T)**2
    return sigma_f**2 * np.exp(-dists / (2*h**2))

def rbf_kernel3(x, z, h=7, sigma_f=1.5):
    x = x.reshape(-1,1)
    z = z.reshape(-1,1)
    dists = (x - z.T)**2
    return sigma_f**2 * np.exp(-dists / (2*h**2))

# Constant mean
def const_mean(x, c=5.0):
    return c * np.ones(len(x))

# Call function for men's marathon
gp_olympic_event(year, mpm, rbf_kernel3, const_mean, noise=(5/60)**2,
                  event_name="Men's marathon min/mile")

# Call function for women's marathon
gp_olympic_event(wyear, wmpm, rbf_kernel2, const_mean, noise=(5/60)**2,
                  event_name="Women's marathon min/mile")

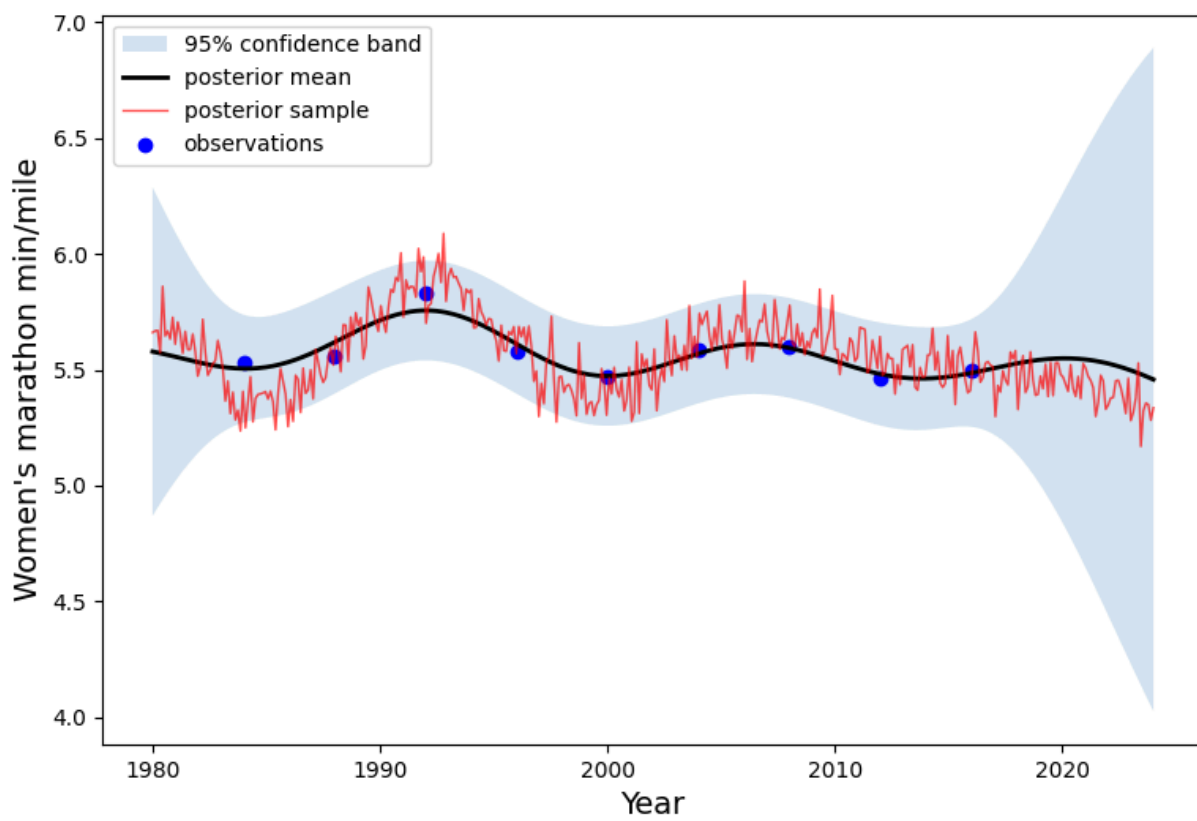
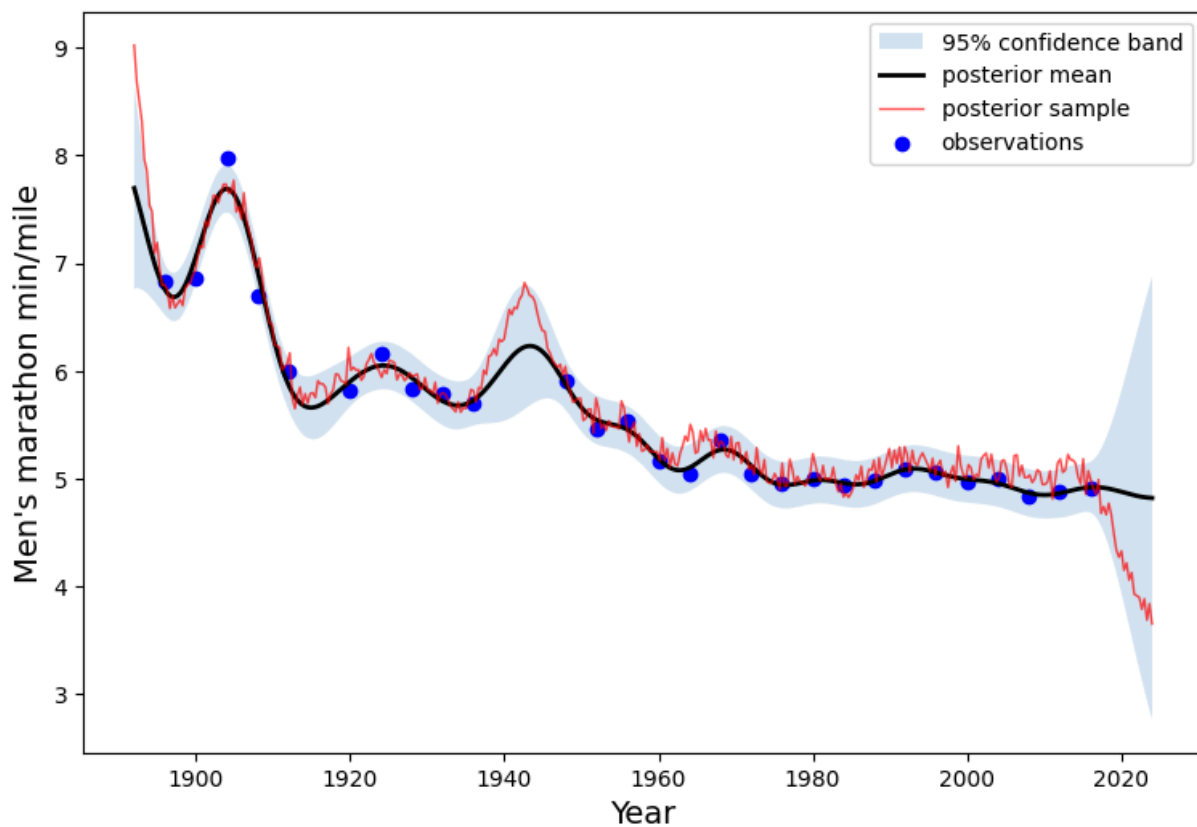
# Men's 100M
gp_olympic_event(m100_year, m100_time, rbf_kernel1, const_mean, noise=(5/60)
                  event_name="Mens's 100M")

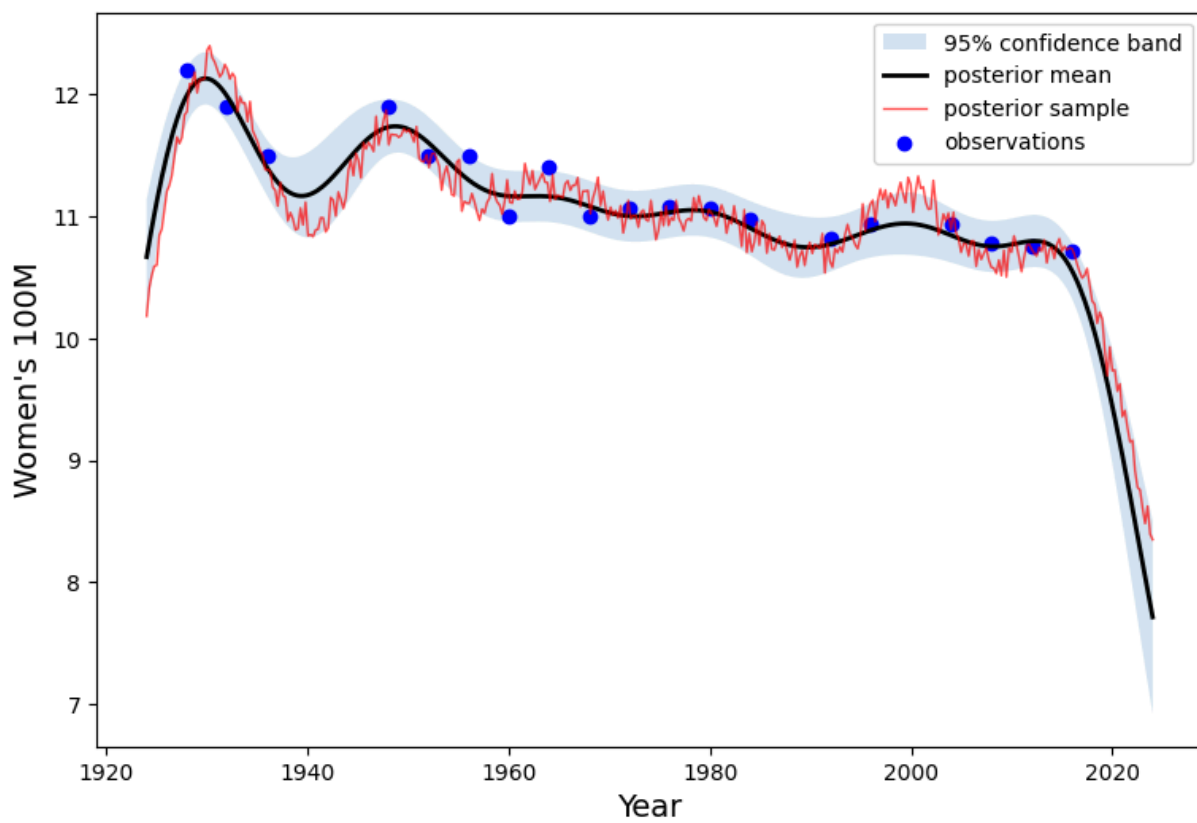
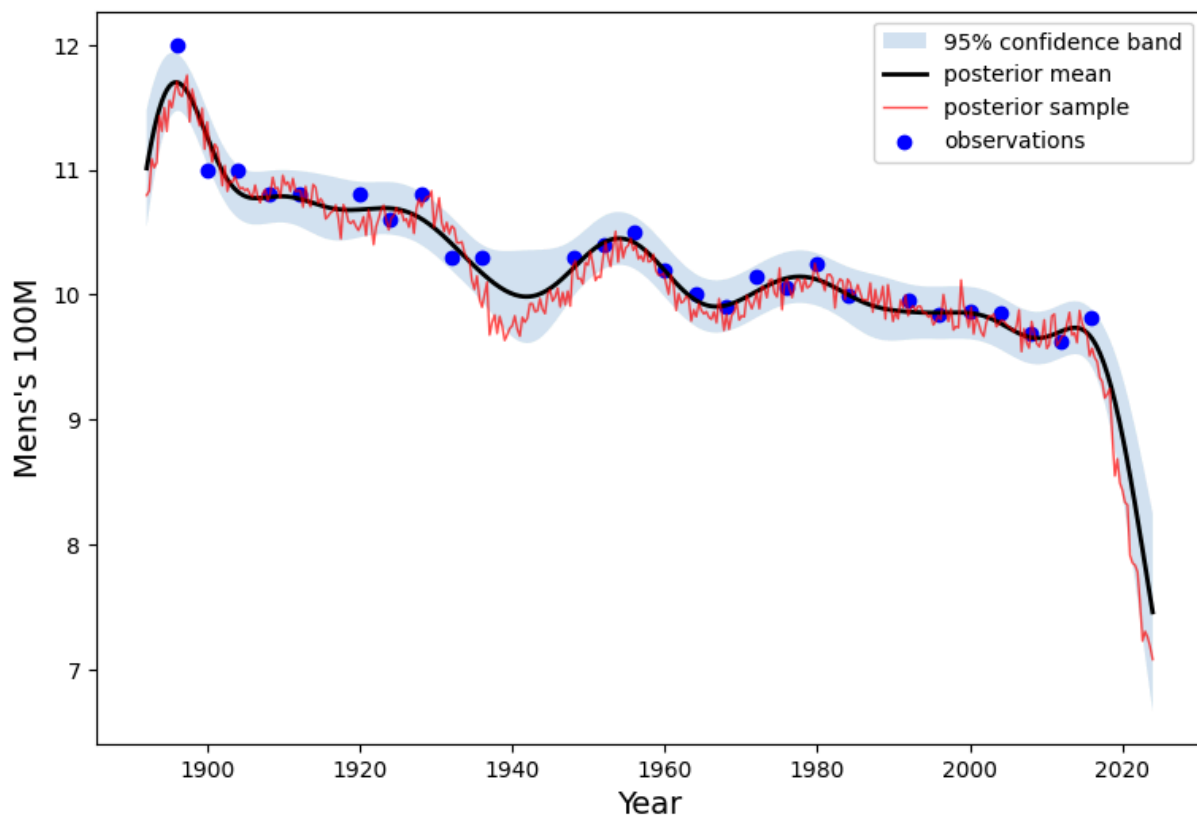
# Women's 100M
gp_olympic_event(w100_year, w100_time, rbf_kernel1, const_mean, noise=(5/60)
                  event_name="Women's 100M")

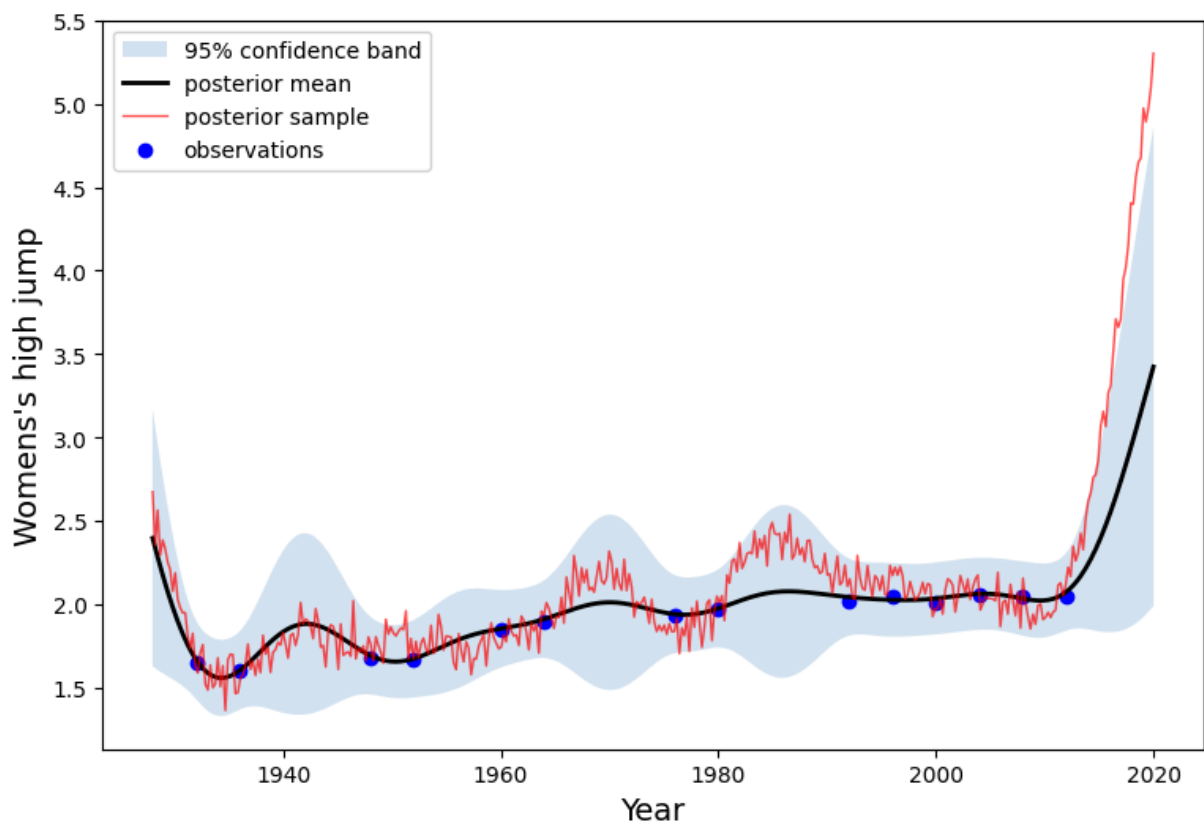
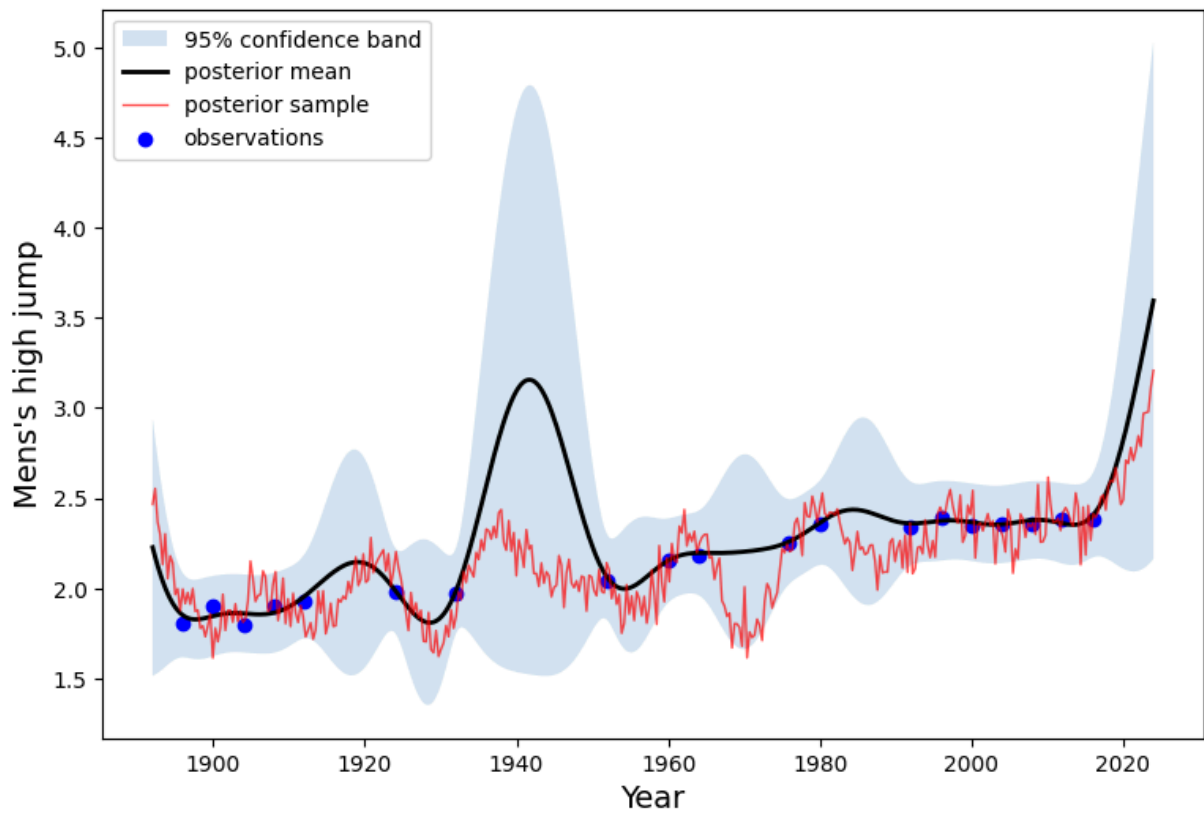
# Men's high jump
gp_olympic_event(mhigh_year, mhigh_height, rbf_kernel2, const_mean, noise=(5
                  event_name="Mens's high jump")

# Women's high jump
gp_olympic_event(whigh_year, whigh_height, rbf_kernel2, const_mean, noise=(5
                  event_name="Womens's high jump")

```







Why I picked the kernel properties that I did: I messed around a bit with the kernel function, and I found that I was able to get a better fit mostly by changing the kernel variance sigma (for example, in the men's marathon, I found that $\sigma = 1.5$ seemed to fit well).

Why I picked the events I did: I picked these Olympic events mostly randomly based on what I found interesting.

Gaps in the data: We see that the gaps in the data lead to quite wide posterior distribution, which makes sense.

Notable events: As we saw in class, the 1904 men's marathon seems like an outlier. Other outliers include the Men's 100M and perhaps the 1948 Women's 100M.

Problem 3: Double descent! (20 points)



In this problem you will explore the "double descent" phenomenon that was recently discovered as a key principle underlying the performance of deep neural networks. The problem setup is a "random features" version of a 2-layer neural network. The weights in the first layer are random and fixed, and the weights in the second layer are estimated from data. As we increase the number of neurons in the hidden layer, the dimension p of

model increases. It's helpful to define the ratio $\gamma = p/n$ of variables to sample points. If $\gamma < 1$ then we want to use the OLS estimator, and if $\gamma > 1$ we want to use the minimum norm estimator.

Your mission (should you choose to accept it), is

1. Implement a function `OLS_or_minimum_norm` that computes the least squares solution when $\gamma < 1$, and the minimum norm solution when $\gamma > 1$. (When $\gamma = 1$ the estimator does not, in general, exist.)
2. Run the main code we give you to average over many trials, and to compute and plot the estimated risk for a range of values of γ .
3. Next, extend the starter code so that you compute (estimates of) the squared-bias and variance of the models. To do this, note that you'll need access to the true regression function, which is provided. You may want to refer to the demo code for smoothing kernels as an example.
4. Using your new code, extend the plotting function we provide so that you plot the squared-bias, variance, and risk together on the same plot.
5. Finally, comment on the results, describing why it might make sense that the squared bias, variance, and risk have the given shapes that they do.
6. Show that in the overparameterized regime $\gamma > 1$, as $\lambda \rightarrow 0$, the ridge regression estimator converges to the minimum norm estimator.

By doing this exercise you will solidify your understanding of the meaning of bias and variance, and also gain a better understanding of the "double descent" phenomenon for overparameterized neural networks, and their striking resistance to overfitting.

We're available in OH to help with any issues you run into!

If you have any interest in background reading on this topic (not expected or required), we recommend Hastie et al., ["Surprises in high-dimensional ridgeless least squares regression"](#).

```
In [16]: import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline
```

Problem 3.1

Implement the function `OLS_or_minimum_norm` that computes the OLS solution for $\gamma < 1$, and the minimum norm solution for $\gamma > 1$.

```
In [17]: def OLS_or_minimum_norm(X, y):
        """
        Return the estimator beta for the random-features regression:
        - If  $\gamma = p/n < 1$  and X has full column rank -> ordinary least squares
        - If  $\gamma = p/n > 1$  -> minimum-norm interpolating solution.
        Notes:
        - Implemented via SVD for numerical stability.
        - At  $\gamma \approx 1$  (or if rank deficiency violates the case assumptions), we r
        """
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float).reshape(-1)
        n, p = X.shape
        gamma = p / n

        # SVD
        U, s, Vt = np.linalg.svd(X, full_matrices=False)
        # Rank test tolerance (standard LAPACK-style)
        tol = np.finfo(float).eps * max(n, p) * (s[0] if s.size else 0.0)
        rank = np.sum(s > tol)

        if gamma < 1:
            # OLS exists only if full column rank
            if rank < p:
                raise ValueError("OLS not unique: X is rank-deficient with p>n.")
            # OLS via SVD:  $V \Sigma^{-1} U^T y$ 
            beta = Vt.T @ ((y @ U) / s)
            return beta

        if gamma > 1:
            # Minimum-norm interpolating solution (pseudoinverse yields min-norm
            beta = Vt.T @ ((y @ U) / s)
            return beta
```

```
# gamma == 1 (in general no solution) or ambiguous case
raise ValueError("Estimator may not exist at  $\gamma = 1$ ; use ridge regularization")
```

In [18]: *# A plotting function we provide. No need to change this, although you can i*

```
def plot_double_descent_risk(gammas, risk, sigma):
    gammas = np.round(gammas, 2)
    fig, ax = plt.subplots(figsize=(10,6))
    tick_pos = np.zeros(len(gammas))
    for i in np.arange(len(gammas)):
        if gammas[i] <= 1:
            tick_pos[i] = gammas[i] * 10
        else:
            tick_pos[i] = gammas[i] + 9
    ax.axvline(x=tick_pos[np.array(gammas)==1][0], linestyle='dashed', color='gray')
    ax.axhline(y=sigma**2, linestyle='dashed', color='gray')
    ax.scatter(tick_pos, risk, color='salmon')
    ax.plot(tick_pos, risk, color='gray', linewidth=.5)

    tickgam = [gam for gam in gammas if (gam > .05 and gam <= .9) or gam >= 1]
    ticks = [tick_pos[j] for j in np.arange(len(tick_pos)) if gammas[j] in tickgam]
    ax.xaxis.set_ticks(ticks)
    ax.xaxis.set_ticklabels(tickgam)

    plt.xlabel(r'$\gamma = \frac{p}{n}$', fontsize=18)
    _ = plt.ylabel('Risk', fontsize=18)
```

Data setup

The following cell sets up our data. The inputs X are random Gaussian vectors of dimension $d = 10$. Then, we map these using a neural network with fixed, Gaussian weights, to get random features corresponding to $p^* = 150$ hidden neurons. The second layer coefficients are $\beta^* \in \mathbb{R}^{p^*}$, which are fixed. This defines the true model.

In [19]: *# just execute this cell, after you define the function above.*

```
np.random.seed(123456)

sigma = 1
d = 10
p_star = 150
signal_size = 5

W_star = (1/np.sqrt(d)) * np.random.randn(d, p_star)
beta_star = np.arange(p_star)
beta_star = signal_size * beta_star / np.sqrt(np.sum(beta_star**2))

N = 10000
X = np.random.randn(N, d)

# f_star is the true regression function, for computing the squared bias
f_star = np.dot(np.tanh(np.dot(X, W_star)), beta_star)
```

```
noise = sigma * np.random.randn(N)
y = f_star + noise
yf = np.concatenate((y.reshape(N,1), f_star.reshape(N,1)), axis=1)
```

Train a sequence of models for different values of γ

Next, we train a sequence of models for different values of γ , always fixing the sample size at $n = 200$, but varying the dimension $p = \gamma n$. When $p < p^*$ we just take the first p features in the true model. When $p > p^*$ we add $p - p^*$ neurons to the hidden layer, with their own random weights.

In the code below, we loop over the different values of γ , and for each γ we run 100 trials, each time generating a new training set of size $n = 200$. The model (either OLS or minimum norm) is then computed, the MSE is computed, and finally the risk is estimated by averaging over all 100 trials.

```
In [20]: trials = 100
n = 200

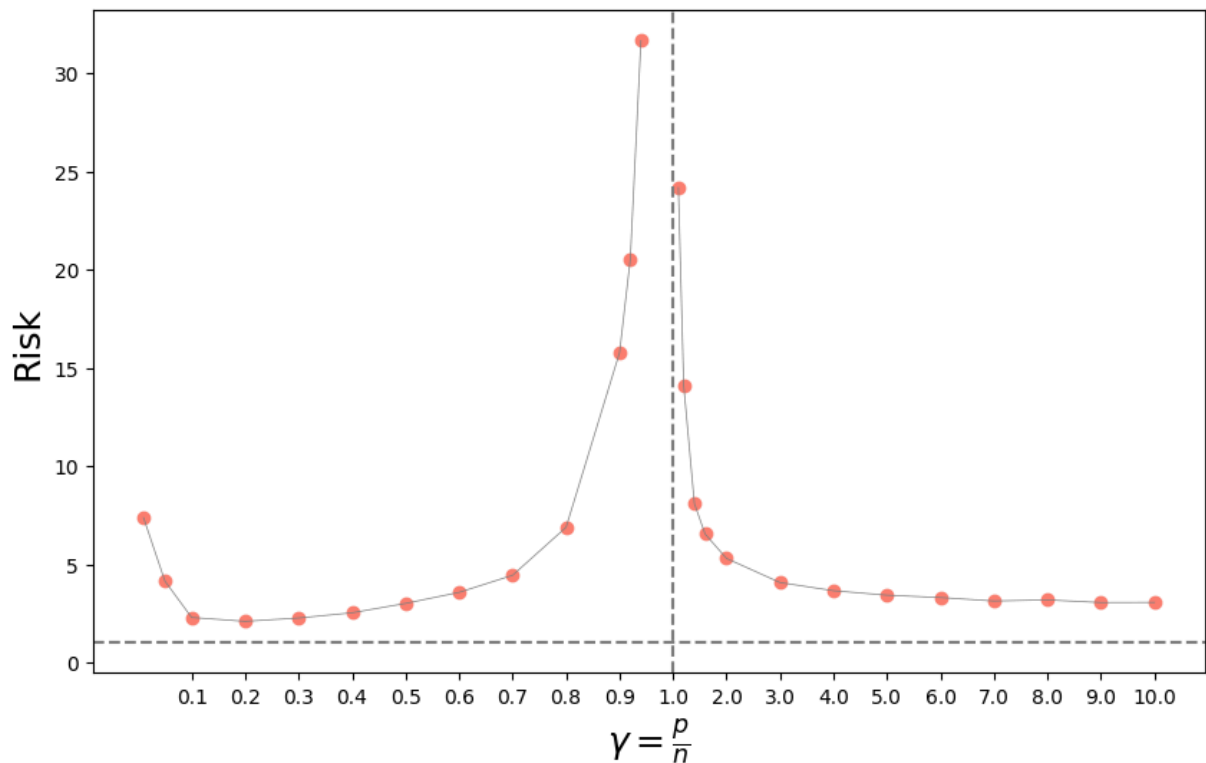
gammas = list(np.arange(.1, 1, .1)) + [.92, .94, 1, 1.1, 1.2, 1.4, 1.6] + li
gammas = [.01, .05] + gammas
risk = []
for gamma in gammas:
    err = []
    p = int(n * gamma)
    if gamma == 1:
        risk.append(np.inf)
        continue
    W = (1/np.sqrt(d)) * np.random.randn(d, p)
    W[:, :min(p, p_star)] = W_star[:, :min(p, p_star)]
    for i in np.arange(trials):
        X_train, X_test, yf_train, yf_test = train_test_split(X, yf, train_s
        H_train = np.tanh(np.dot(X_train, W))
        H_test = np.tanh(np.dot(X_test, W))
        beta_hat = OLS_or_minimum_norm(H_train, yf_train[:,0])
        yhat_test = H_test @ beta_hat
        err.append(np.mean((yhat_test - yf_test[:,0])**2))
    print('gamma=%.2f p=%d n=%d risk=%.3f' % (gamma, p, n, np.mean(err)))
    risk.append(np.mean(err))
```


gamma=0.01	p=2	n=200	risk=7.373
gamma=0.05	p=10	n=200	risk=4.146
gamma=0.10	p=20	n=200	risk=2.294
gamma=0.20	p=40	n=200	risk=2.099
gamma=0.30	p=60	n=200	risk=2.267
gamma=0.40	p=80	n=200	risk=2.534
gamma=0.50	p=100	n=200	risk=3.011
gamma=0.60	p=120	n=200	risk=3.573
gamma=0.70	p=140	n=200	risk=4.443
gamma=0.80	p=160	n=200	risk=6.885
gamma=0.90	p=180	n=200	risk=15.773
gamma=0.92	p=184	n=200	risk=20.500
gamma=0.94	p=188	n=200	risk=31.661
gamma=1.10	p=220	n=200	risk=24.179
gamma=1.20	p=240	n=200	risk=14.097
gamma=1.40	p=280	n=200	risk=8.089
gamma=1.60	p=320	n=200	risk=6.543
gamma=2.00	p=400	n=200	risk=5.301
gamma=3.00	p=600	n=200	risk=4.073
gamma=4.00	p=800	n=200	risk=3.658
gamma=5.00	p=1000	n=200	risk=3.430
gamma=6.00	p=1200	n=200	risk=3.309
gamma=7.00	p=1400	n=200	risk=3.135
gamma=8.00	p=1600	n=200	risk=3.190
gamma=9.00	p=1800	n=200	risk=3.056
gamma=10.00	p=2000	n=200	risk=3.062

Plot the risk

At this point, you can plot the risk by just evaluating the cell below. This should reveal the "double descent" behavior.

```
In [21]: # Just evaluate the next line
plot_double_descent_risk(gammas, risk, sigma)
```



Problem 3.2

Comment on the results. Explain why the risk plot does or does not make sense in each regime: The underparameterized regime $\gamma < 1$, and the overparameterized regime $\gamma > 1$. Is the curve "U-shaped" in the underparameterized regime? Why or why not? What about in the overparameterized regime? You will be able to give better answers to these questions when you estimate the bias and variance below.

Here, we show the classic double descent phenomenon. When gamma is less than one, we are in the world of standard regression, where adding parameters first drops (squared) bias more than it raises variance, so the risk goes down. However, eventually the gain in variance outweighs the loss in bias, so risk goes up as gamma gets close to one. Then, we see that when gamma is greater than one and we switch to the minimum norm estimator, the risk actually starts going down again. We showed that the minimum norm is akin to regularization, so we prefer smaller weights, taming variance, thus reducing the overall risk once more.

Problem 3.3

Now, modify the above code so that you can estimate both the squared bias and the variance of the estimator. Before you do this, you may want to revisit the kernel smoothing demo from class, where we computed the squared bias, variance, and risk. You'll need the true function, which is provided in the variable `yf`. You should not have

to write a lot of code, but can compute the bias and variance after you store the predicted values on the test data for each trial.

Plot the results, by plotting both the squared bias, the variance, and the risk for the sequence of gammas. To do this you will have to modify the plotting function appropriately, but this again involves minimal changes. When you obtain your final plot, comment on the shape of the bias and variance curves, as above for Problem 3.2.

```
In [22]: # --- config ---
        trials = 100
        n = 200
        TEST_SZ = 2000 # fixed test set for bias/variance estimation
        rng = np.random.default_rng(2025)

        # reuse your gamma grid
        gammas = list(np.arange(.1, 1, .1)) + [.92, .94, 1, 1.1, 1.2, 1.4, 1.6] + li
        gammas = [.01, .05] + gammas

        # ---- helper: tick mapping identical to your risk plot ----
        def _tick_positions(gammas_arr):
            gammas_arr = np.round(np.array(gammas_arr, dtype=float), 2)
            tick_pos = np.zeros_like(gammas_arr, dtype=float)
            for i in range(len(gammas_arr)):
                tick_pos[i] = gammas_arr[i]*10 if gammas_arr[i] <= 1 else gammas_arr[i]
            return gammas_arr, tick_pos

        # ---- new plot: Bias^2, Var, Risk on one axis (uses same tick mapping idea)
        def plot_bias_variance_risk(gammas, risk, bias2, var, sigma):
            gammas_arr, tick_pos = _tick_positions(gammas)
            fig, ax = plt.subplots(figsize=(10,6))

            # reference lines
            ax.axvline(x=tick_pos[np.where(gammas_arr==1)][0], linestyle='dashed', color='gray', label=r'noise sigma')
            ax.axhline(y=sigma**2, linestyle='dashed', color='gray', label=r'noise sigma^2')

            # curves
            ax.plot(tick_pos, bias2, marker='o', ms=4, lw=1.5, label='Bias$^2$', color='blue')
            ax.plot(tick_pos, var, marker='o', ms=4, lw=1.5, label='Variance', color='red')
            ax.plot(tick_pos, risk, marker='o', ms=4, lw=1.5, label='Risk (MSE)', color='green')

            # ticks, labels (same selection rule)
            tickgam = [gam for gam in gammas_arr if (gam > .05 and gam <= .9) or gam == 1]
            ticks = [tick_pos[j] for j in range(len(tick_pos)) if gammas_arr[j] in tickgam]
            ax.set_xticks(ticks); ax.set_xticklabels(tickgam)

            ax.set_xlabel(r'$\gamma = \frac{p}{n}$', fontsize=18)
            ax.set_ylabel('Error components', fontsize=18)
            ax.legend()
            plt.show()
```

```
In [23]: trials = 100
        n = 200

        gammas = list(np.arange(.1, 1, .1)) + [.92, .94, 1, 1.1, 1.2, 1.4, 1.6] + li
```

```

gammas = [.01, .05] + gammas

risk = []
bias2 = []
var = []

for gamma in gammas:
    p = int(n * gamma)
    if gamma == 1:
        risk.append(np.inf)
        bias2.append(np.nan)
        var.append(np.nan)
        continue

    W = (1/np.sqrt(d)) * np.random.randn(d, p)
    W[:, :min(p, p_star)] = W_star[:, :min(p, p_star)]

    preds_all = []
    errs = []

    for i in np.arange(trials):
        X_train, X_test, yf_train, yf_test = train_test_split(X, yf, train_size=train_size)
        H_train = np.tanh(np.dot(X_train, W))
        H_test = np.tanh(np.dot(X_test, W))

        beta_hat = OLS_or_minimum_norm(H_train, yf_train[:,0])
        yhat_test = H_test @ beta_hat

        preds_all.append(yhat_test)
        errs.append(np.mean((yhat_test - yf_test[:,0])**2))

    preds_all = np.array(preds_all)          # shape (trials, test_size)
    f_true = yf_test[:,1]                   # true regression values on test set
    f_bar = preds_all.mean(axis=0)          # average predictor across trials

    # compute bias^2 and variance
    bias2_g = np.mean((f_bar - f_true)**2)
    var_g = np.mean(preds_all.var(axis=0, ddof=1))

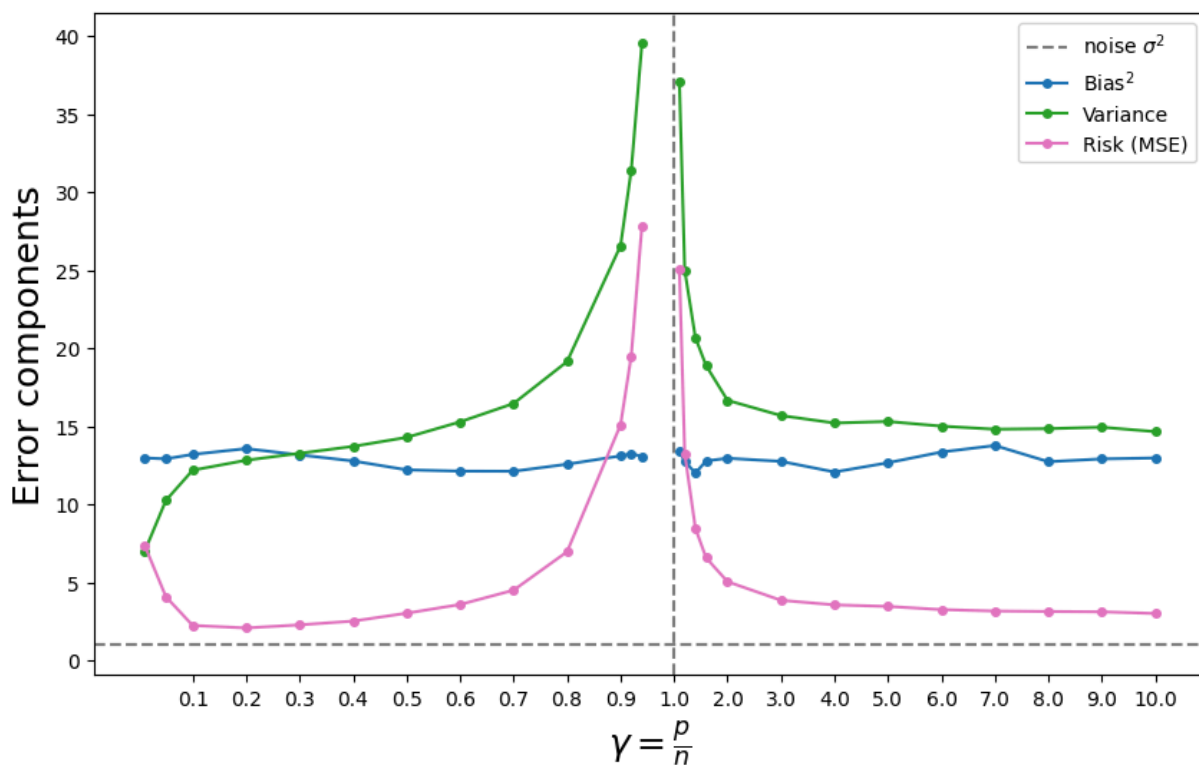
    risk.append(np.mean(errs))
    bias2.append(bias2_g)
    var.append(var_g)

    print(f'gamma={gamma:.2f}  p={p}  n={n}  risk={np.mean(errs):.3f}  bias^2={bias2_g:.3f}  var={var_g:.3f}')

```

gamma	p	n	risk	bias^2	var
gamma=0.01	p=2	n=200	risk=7.407	bias^2=12.981	var=6.984
gamma=0.05	p=10	n=200	risk=4.095	bias^2=12.926	var=10.265
gamma=0.10	p=20	n=200	risk=2.259	bias^2=13.211	var=12.198
gamma=0.20	p=40	n=200	risk=2.097	bias^2=13.581	var=12.835
gamma=0.30	p=60	n=200	risk=2.295	bias^2=13.177	var=13.280
gamma=0.40	p=80	n=200	risk=2.536	bias^2=12.791	var=13.723
gamma=0.50	p=100	n=200	risk=3.040	bias^2=12.222	var=14.299
gamma=0.60	p=120	n=200	risk=3.599	bias^2=12.137	var=15.294
gamma=0.70	p=140	n=200	risk=4.515	bias^2=12.130	var=16.470
gamma=0.80	p=160	n=200	risk=6.976	bias^2=12.582	var=19.151
gamma=0.90	p=180	n=200	risk=15.100	bias^2=13.115	var=26.571
gamma=0.92	p=184	n=200	risk=19.507	bias^2=13.244	var=31.393
gamma=0.94	p=188	n=200	risk=27.844	bias^2=13.094	var=39.518
gamma=1.10	p=220	n=200	risk=25.109	bias^2=13.422	var=37.119
gamma=1.20	p=240	n=200	risk=13.271	bias^2=12.763	var=25.008
gamma=1.40	p=280	n=200	risk=8.477	bias^2=12.018	var=20.680
gamma=1.60	p=320	n=200	risk=6.588	bias^2=12.792	var=18.894
gamma=2.00	p=400	n=200	risk=5.061	bias^2=12.964	var=16.673
gamma=3.00	p=600	n=200	risk=3.868	bias^2=12.764	var=15.698
gamma=4.00	p=800	n=200	risk=3.576	bias^2=12.083	var=15.220
gamma=5.00	p=1000	n=200	risk=3.475	bias^2=12.675	var=15.318
gamma=6.00	p=1200	n=200	risk=3.274	bias^2=13.362	var=15.020
gamma=7.00	p=1400	n=200	risk=3.180	bias^2=13.779	var=14.822
gamma=8.00	p=1600	n=200	risk=3.152	bias^2=12.741	var=14.859
gamma=9.00	p=1800	n=200	risk=3.131	bias^2=12.922	var=14.952
gamma=10.00	p=2000	n=200	risk=3.024	bias^2=12.981	var=14.667

```
In [24]: # Print plot with added features
plot_bias_variance_risk(gammas, risk, bias2, var, sigma)
```



This confirms the predictions that I mentioned above. We see that, as gamma goes above 1, the variance is what changes. Bias, on the other hand, remains at about the

same (low) level. Variance quickly begins to fall again, reaching a more stable lower level by about $\gamma = 3.0$. In this region, the minimum norm solution begins to act akin to regularization.

Problem 3.4

In class, we discussed the interpretation of the minimum-norm estimator $\hat{\beta}_{\text{mn}}$.

Geometrically, we can describe $\hat{\beta}_{\text{mn}}$ as the orthogonal projection of the zero vector in $\mathbb{R}^p$ onto the $(p - 1)$ -dimensional hyperplane $\{\beta : X\beta = Y\}$.

This can also be viewed as "ridgeless" regression. In ridge regression, we minimize the objective function

$$\|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2,$$

which has the closed-form solution

$$\hat{\beta}_\lambda = (X^T X + \lambda I)^{-1} X^T Y.$$

In the overparameterized regime where $p > n$, it can be shown that as $\lambda \rightarrow 0$, $\hat{\beta}_\lambda$ converges to $\hat{\beta}_{\text{mn}}$.

Your task is to show that as $\lambda \rightarrow 0$, the limit of the ridge regression estimator $\hat{\beta}_\lambda$, in the overparameterized regime where $\gamma > 1$, is the minimum-norm estimator $\hat{\beta}_{\text{mn}}$. You may want to use the Woodbury formula for this derivation.

Hint:

1. Applying the simplified version of Woodbury formula

$$(I + UV^T)^{-1} = I - U(I + V^T U)^{-1} V^T.$$

we can derive the identity:

$$(X^T X + \lambda I_p)^{-1} X^T = X^T (X X^T + \lambda I_n)^{-1},$$

2. You might consider using the Woodbury formula twice.

3.4

Ridge regression: $\min_b \{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \}$

$$\Rightarrow \hat{\beta}_\lambda = (X'X + \lambda I_p)^{-1} X'y$$

Minimum Norm: $\hat{\beta}_{mn} = \arg\min_{\beta} \|\beta\|_2^2 \text{ s.t. } X\beta = y$

$$\Rightarrow \hat{\beta}_{mn} = X'(XX')^{-1}y \quad (\text{which we showed in class})$$

$$\hat{\beta}_\lambda = (X'X + \lambda I_p)^{-1} X'y$$

$$= X'(XX' + \lambda I_n)^{-1}y \quad \text{by Woodbury formula}$$

$$\Rightarrow \lim_{\lambda \rightarrow 0^+} \hat{\beta}_\lambda = X'(XX')^{-1}y = \hat{\beta}_{mn}$$